GE VERNOVA

ADMS

16.25.1

Archive User Guide

Copyright

Change History

Date	Change Description
Jan 30, 2023	ADMS 3.16: • Updated Devices. • Updated Enterprise OMS Data. • Updated Coded Translation Data. • Editorial and formatting updates.
Oct 20, 2023	ADMS 16.1.0: • The product version numbering has changed. The prefix '3' is no longer included in the version numbering for ADMS.
Apr 4, 2024	ADMS 16.7.0: • Updated document format and styling.
Apr 21, 2025	ADMS 16.17.0: • Updated Coded Translation Data.

Contents

1. Introduction

The ADMS Archive database contains historical data from the Distribution Management System, Outage Management System, Switching Order, and Safety Document applications for reporting purposes. The database stores Distribution Power Flow (DPF) solutions (summary data only), "advanced application" (FISR, LVM, AFR) solutions, incidents, crews, calls, meter events, meter statuses, switching orders, safety documents, customer data, and reference data. Some company-specific static data (for example, cities, service centers, etc.) are loaded manually.

The ADMS Archive database is loaded by the ADMS Archive application. Customer data is loaded by the Customer Loader tool. After the Archive application is configured, it runs as a background process on a server and requires no user interaction. The Archive application automatically records data from journals and some other sources, such as Safety Documents files, in the database.

The Archive application processes data from different sources independently in parallel. Data from each source is archived sequentially based on data timestamp.

1.1. Installing the Archive Application

The ADMS Archive application stores various types of historical data in an Oracle or a MS SQL Server relational database.

The Archive application is included in the ADMS server installation kit. If you want to run the Archive application on the same application server as other server applications, no additional installation steps are necessary. However, if you want to run the Archive application on a different application server, such as the SCADA server, you must copy the following Archive application assembly files to a folder (for example, `D:\eterra\distribution\Archive\bin`) on the Archive application server:

- ArchiveApp.exe
- ArchiveApp.resources.dll
- ArchiveDataAccess.dll
- ArchiveDataModel.dll
- ArchiveUtilities.dll
- IDMSTokenProvider.dll
- NetUtil.dll
- NetUtil.resources.dll
- OptUtil.dll
- ProductProperties.dll
- SerializedFile.dll

> **Note**
>
> To run the Archive application on the SCADA server, the SCADA server must be running on

> Windows.

In addition, three "entity data model metadata" files are required. These metadata files are used by Entity Framework within the Archive application. They describe the object model, database schema, and the mapping between the object model and database schema. For an Oracle database, the files are named `Archive_Oracle.csdl`, `Archive_Oracle.ssdl`, and `Archive_Oracle.msl`. For a SQL Server database, the files are named `Archive_SqlServer.csdl`, `Archive_ SqlServer.ssdl`, and `Archive_SqlServer.msl`.

The Oracle Data Provider for .NET (ODP.NET) assemblies, `Oracle.ManagedDataAccess.dll` and `Oracle.ManagedDataAccess.EntityFramework.dll` (or `Oracle.DataAccess.dll` and `Oracle.DataAccess.EntityFramework.dll`, if you are using ODP.NET Unmanaged Driver), must also exist on the Archive application server. They may be installed in the ODP.NET installation folder or may exist in the folder containing the Archive application assemblies.

Note that the Archive application can run on multiple application servers. Consider, for example, a configuration in which the Safety Document application runs on the SCADA application server, and the OMS and Switching Order applications run on the OMS application server. In such a case, you may need to have one Archive application instance running on the SCADA application server to archive safety documents, and another Archive application instance running on the OMS application server to archive incidents and switching orders. This avoids the need for safety document Archive files and Archive journal files to be transferred between application servers. The multiple Archive application instances may update the same Archive database.

If the Archive application runs only on an application server other than the main server application server, consider how reference data is updated in the Archive database. The Archive application loads some static reference data (for example, device types) from the `CodedTranslationConfig.xml` file, which is updated by the Coded Translation Configuration Editor tool and used by both the server and client applications. To load updated reference data into the Archive database, updated `CodedTranslationConfig.xml` files must be accessible to the Archive application. They will be accessible if the Archive application runs on the same application server as the other server applications. However, if the Archive application runs on a different application server, File Updater or a manual process may be necessary to copy the updated `CodedTranslationConfig.xml` files to the Archive application server. The path to the `CodedTranslationConfig.xml` file is specified in the Archive application's configuration file, `ArchiveAppConfig.xml`.

1.2. Installing Oracle Data Provider for .NET (ODP.NET)

The ADMS Archive and DNOM Database Loader applications store data in relational databases. Supported relational database management systems include MS SQLServer and Oracle. Data access is via Entity Framework and a data provider. The data provider for SQL Server is Microsoft .NET Framework Data Provider for SQL Server (SqlClient). SqlClient is part of the Microsoft .NET Framework and therefore does not require a separate installation. The data provider for Oracle is Oracle Data Provider for .NET (ODP.NET). ODP.NET must be installed separately. Thus, if your ADMS configuration includes the Archive or DNOM Database Loader application and an Oracle database, you must install ODP.NET on the servers on which the Archive or DNOM Database Loader

applications run. You should also install Oracle Instant client and SQL*Plus for testing database connectivity without running the ADMS applications.

Although ODP.NET, Oracle Instant client, and SQL*Plus are free, GE is not authorized to redistribute Oracle software. You must download them from the Oracle Technology Network at the following URL: http://www.oracle.com/technetwork/topics/dotnet/index-085163.html

ODP.NET, Oracle Instant client, and SQL*Plus are included in the Oracle Data Access Components (ODAC) download. For installation instructions, refer to the ODAC documentation. Since ODAC installation loads assemblies into the .NET Global Assembly Cache (GAC), you must uninstall older versions of these ODAC components before installing new versions to prevent the possibility of using old versions at run time. ODAC components may be installed either via Oracle Universal Installer (OUI) or via command scripts in the Xcopy distributions. You do not need to install other ODAC components such as Oracle Data Providers for `ASP.NET`.

ADMS Archive supports ODP.NET 19.7. Older versions of ODP.NET, including ODP.NET 11g (11.2) and ODP.NET 12.2c (12.2) are not supported.

ODP.NET includes both a Managed Driver and an Unmanaged Driver. Although ADMS Archive works with either driver, it is configured for the Managed Driver. The default configuration could be overridden via an application configuration file (`ArchiveApp.exe`.config), but since the Unmanaged Driver provides no functionality required by Archive that is not provided by the Managed Driver, an ArchiveApp.exe.config file is not supplied in ADMS releases. This documentation focuses on the Managed Driver.

The Entity Framework metadata file Archive_Oracle.ssdl included in this release of ADMS is configured for ODP.NET, Managed Driver and an Oracle Database 12c database. Ifyou are using ODP.NET, Unmanaged Driver, or a different version of Oracle Database, minor changes must be made to the metadata file. For more information, refer to the explanation of the DataProviderName configuration parameter in DataProviderName.

The ODP.NET, Managed Driver main assembly is `Oracle.ManagedDataAccess.dll`. In ODP.NET, Oracle.ManagedDataAccess.EntityFramework.dll supports Entity Framework 6. Properties of these files are shown in the below images.

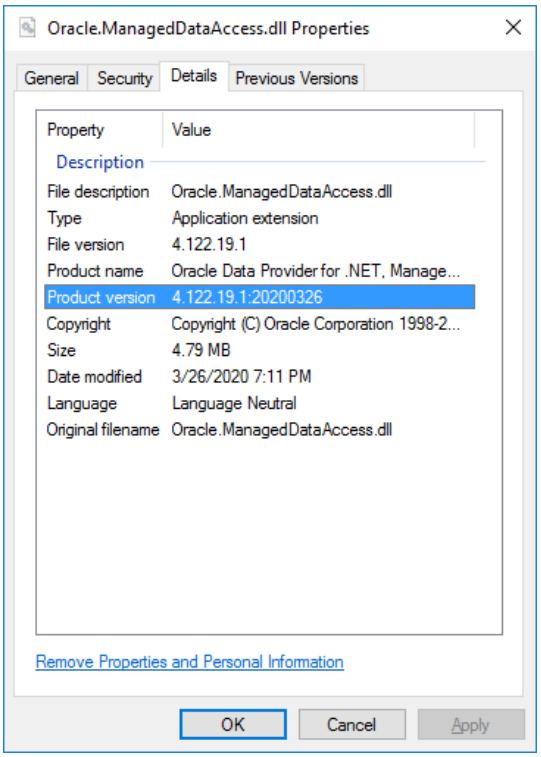


Figure 1. Oracle.ManagedDataAccess.dll Properties

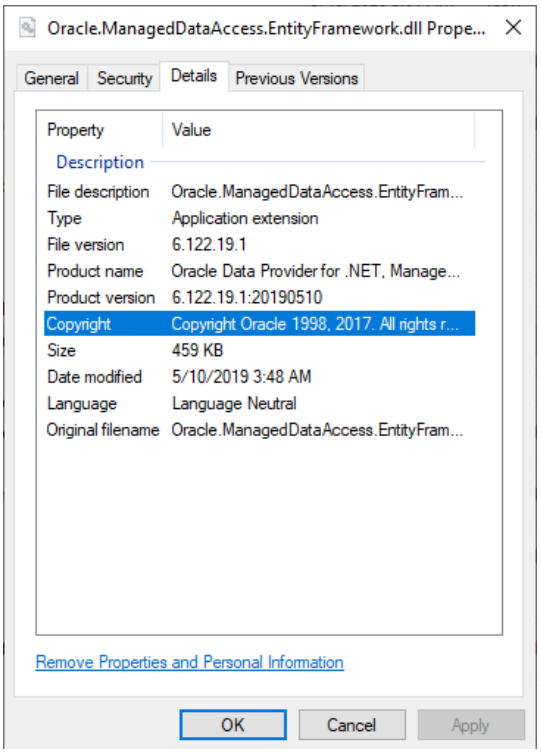


Figure 2. Oracle.ManagedDataAccess.EntityFramework.dll Properties

Since ADMS Archive uses Entity Framework 6, it requires `Oracle.ManagedDataAccess.EntityFramework.dll` for the Managed Driver. One option to enable this

assembly to load is to copy `Oracle.ManagedDataAccess.EntityFramework.dll` to the bin directory that contains the ADMS Archive assemblies (for example, `D:\Eterra\Distribution\Server\bin`). Alternatively, you may install the assembly in the GAC as follows:

1. Open a Command Prompt window in the folder that contains gacutil.exe (for example, "C:\Program Files (x86)\Microsoft SDKs\Windows\v8.1A\bin\NETFX 4.5.1 Tools\x64").
2. Enter the following command, where <*assembly_path*> is the full path of `Oracle.ManagedDataAccess.EntityFramework.dll`.

```
gacutil /i <assembly_path>
```

2. Archive Database

The ADMSArchive database is a relational database with data organized in standard relational database objects such as tables and indexes. The database is typically installed on a database server that is separate from the ADMS application server. For information about installing or upgrading the database or to view the data dictionary, refer to the *ADMS Relational Database Software Installation and Maintenance Guide.*

The ADMSArchive application is a Microsoft .NET application that reads ADMS data from various sources and stores the data in a relational database. It uses an internal data access library to manage data access in a data provider-independent manner so that the Archive application works with either an Oracle or Microsoft SQL Server database. The Oracle data provider is Oracle Data Provider for .NET (ODP.NET). ODP.NET enables access to an Oracle database from a Microsoft .NET application. ODP.NET in turn uses Oracle Client or Oracle Instant Client to connect to the database. The SQL Server data provider is Microsoft .NET Framework Data Provider for SQL Server (SqlClient). SqlClient, part of the Microsoft .NET Framework, enables access to a SQL Server database from a Microsoft .NET application. For information about installing the Archive application, refer to the *ADMS Software Installation and Maintenance Guide.*

2.1. Database Sources

Although ADMS uses a memory-resident database for operational data, files provide data persistence. This section describes files that are written by other ADMS applications and used as data sources for the Archive database.

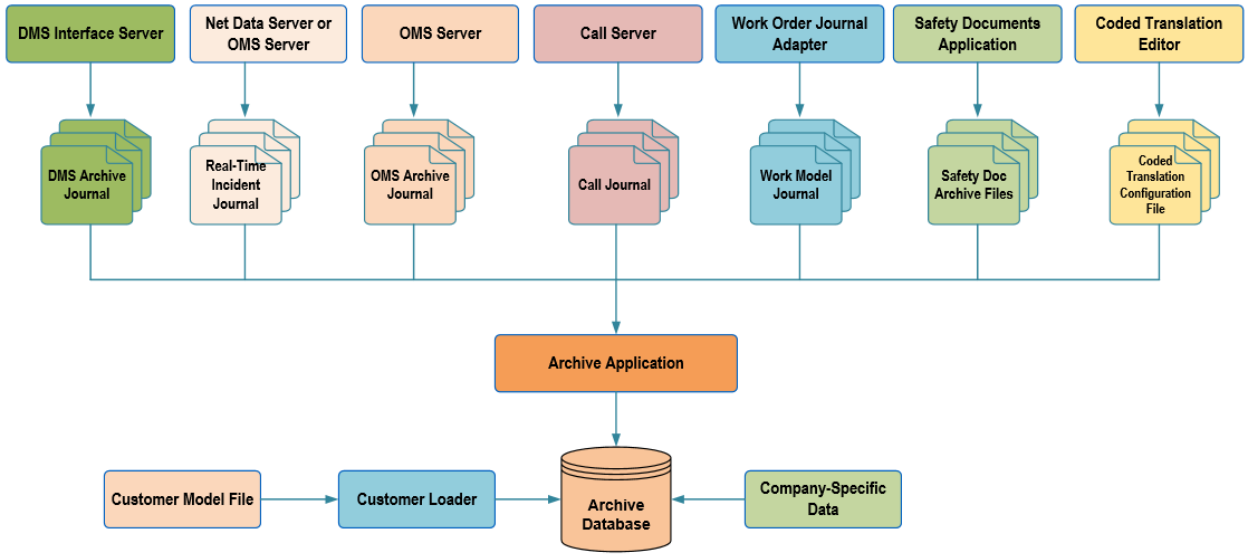


Figure 3. Data Flow Diagram

2.1.1. Journal Files

Journal files are a common type of ADMS file in which data is written chronologically for efficient retrieval of incremental updates. Each journal file has several pre-defined record types. Records are written to journal files as soon as the associated events appear in the system (for example, when an incident is cleared or a DPF solution is generated).

Journals are collections of journal files of specific types. Journal files are archived by the Archive application.

> **Note**
>
> Although different journals may contain similar data, some data may be archived from a specific journal only.

2.1.1.1. DMS Archive Journal

Distribution Management System (DMS) data is written to the DMS archive journal by the DMS Interface server. The DMS Interface server obtains the data to be archived from the DMS server via periodic queries. The query responses are written to the DMS archive journal. The following data types are written to the DMS archive journal.

- Distribution Power Flow Solutions
- Advanced Application Solutions
- Fault Location Solutions
- Station and Feeder Exceptions
- Network Model File Info

2.1.1.2. OMS Archive Journal

Historical incident and switching order data are written to the OMS archive journal by the OMS server (NetServer) when certain events occur, such as when an incident is cleared or canceled, a historical incident is updated, or a switching order is executed. Note that the OMS server writes switching orders to the OMS archive journal, even though switching orders are created by the Switching Order application, which is not part of OMS.

The OMS archive journal contains only historical data, so that it can be queried without impacting operational database performance. The OMS archive journal is periodically queried by the Archive application. Incidents are stored in the Archive database within a few seconds of being written to the OMS archive journal (the OMS archive journal polling interval is configurable).

2.1.1.3. Call Journal

Call Server writes calls, meter events, and meter statuses to a call journal. This data may be stored

in the Archive database. The following types of data are written to the call journal.

- Calls
- Meter Events and Meter Voltage Events
- Meter Statuses and Meter Voltage Statuses (responses to meter pings)

> **Note**
>
> Unlike the DMS and OMS archive journals, which are created specifically for use by the Archive application, the call journal is not used exclusively by the Archive application; it also is used by Call Server for initialization.

2.1.1.4. Real-Time Incident Journal

Near Real-Time (NRT) outage management and switching order data, such as incident, planned outage, and area note information, is written to the real-time incident journal by the Net Data server or OMS server (NetServer) when certain events occur (such as when an incident or an area note is created or updated).

The real-time incident journal is periodically queried by the Archive application (the journal polling interval is configurable). After data is written to the real-time incident journal, the data is stored in the Archive database. The time to write the data to the database greatly varies depending on the number of outages and affected customers. For example, for a million-customer utility, the data is written within a minute for blue-sky conditions and within 5-10 minutes for gray sky (storm) conditions.

> **Note**
>
> If the ADMS Near Real-Time Database is used as the source for outage maps, the outage map queries should be run on a replica of the Near Real-Time Database. This avoids delays in inserting or updating data in the primary Near Real-Time Database.

2.1.1.5. Work Model Journal

Enterprise OMS data is written to the Work Model journal by the Work Order Journal Adapter when certain events occur, such as when an order is created for an incident. Enterprise OMS data may be stored in the Archive database.

2.1.2. Safety Document Archive Files

Safety documents to be archived are not written to a journal. Instead, they are written to XML files, one safety document per file.

2.1.3. CodedTranslationConfig.xml

Coded Translation Data stored in the Archive database is read from the `CodedTranslationConfig.xml` file, which is written by the Coded Translation Editor tool.

2.1.4. Customer Model Files

Customer Data stored in the Archive database is read from a compiled customer file (*.bpcl), which is created by the Customer List Compiler tool.

2.1.5. Company-Specific Information

Company-specific data stored in the Archive database is loaded manually or by custom scripts.

2.2. Database Information

2.2.1. Distribution Power Flow Solutions

The Distribution Power Flow (DPF) application runs almost continuously, producing a large volume of data for each solution. Therefore, limited summary data is archived for each DPF solution. This includes solution time; number of iterations; whether the solution converged; measurement islands; and calculated losses by station, power transformer, and feeder.

The following DMS archive journal record types are archived:

- ArchiveDpfSolution
- ArchiveDpfMeasurementIsland
- ArchiveDpfTfLosses
- ArchiveDpfFeederLosses

This data is stored in the DPF_SOLUTION, DPF_MEASUREMENT_ISLAND, DPF_TF_LOSSES, and DPF_FEEDER_LOSSES tables. Note that calculated losses by station are stored in the DPF_SOLUTION table, since that table contains station-level data, but calculated losses by power transformer and by feeder are stored in separate tables.

2.2.2. Advanced Application Solutions

Archived "advanced application" data includes solutions, plans, and steps from the following Distribution Network Analysis Functions (DNAF) applications:

- Fault Isolation and Service Restoration (FISR)

- Load and Volt/VAr Management (LVM)[1]
- Automatic Feeder Reconfiguration (AFR)

The following DMS archive journal record types are archived:

- ArchiveAdvancedAppSolution
- ArchiveAdvancedAppPlan
- ArchiveAdvancedAppStep
- ArchiveAdvancedAppTfParameters
- ArchiveAdvancedAppFeederParameters
- ArchiveLvmBusVoltage

Advanced application (FISR, LVM, and AFR) solution, plan, and step data are stored in the ADVANCED_APP_SOLUTION, ADVANCED_APP_PLAN, and ADVANCED_APP_STEP tables. Bus voltage data, which is applicable only to LVM plans, is stored in the LVM_BUS_VOLTAGE table.

2.2.3. Fault Location Solutions

Archived Fault Location data includes solutions and results from the Fault Location DNAF application. The following DMS archive journal record types are archived:

- ArchiveFaultLocationSolution
- ArchiveFaultLocationResult

This data is stored in the FAULT_LOCATION_SOLUTION and FAULT_LOCATION_RESULT tables.

2.2.4. Station and Feeder Exceptions

The Network Topology Processor (NTP) application also runs frequently, whenever the network topology changes. Among the most useful information produced by NTP are station and feeder exceptions, which identify stations and feeders with devices in abnormal states. This data appears on the Station Exception and Feeder Exception displays. It is also archived whenever a new DPF solution is available.

The following DMS archive journal record types are archived:

- ArchiveStationException
- ArchiveFeederException

This data is stored in the STATION_EXCEPTION and FEEDER_EXCEPTION tables.

2.2.5. Network Model File Info

The distribution network model is represented by a set of station model files. These station model files are created by the Converter application from a set of input files. When a new station model file is loaded into the Distribution Network Object Model (DNOM), properties (for example, name, creation date, length) of the station model file and the files used to create the station model file are archived, providing an audit trail of the network model files.

The following DMS archive journal record type is archived:

- ArchiveNetworkModelFileInfo

This data is stored in the NETWORK_MODEL_FILE_INFO table.

2.2.6. Outage Management Data

Two types of outage management data are archived:

- Current OMS data (such as created incidents, incident updates, assigned crews, crew model updates, planned outages, area notes, etc.). This data is stored in real-time incident journals.
- Historical incident data (data for cleared and cancelled incidents). This data is stored in OMS archive journals.

The following record types are archived from both the OMS archive journal and the real-time incident journal:

- ArchiveIncident
- ArchiveIncidentDelete
- ArchiveIncidentChronology
- ArchiveRestorationStep
- ArchiveRestorationStepAffectedPremises
- ArchiveRestorationStepSwitchingStep
- ArchiveSwitchingStep
- ArchiveSwitchingStepDelete
- ArchiveRestorationStepSwitchingStep
- ArchiveAssociateSwitchingStep
- ArchiveCustomerCount
- ArchiveDamageAssessmentLocation
- ArchiveDeleteAssessmentLocation
- ArchiveDACustomField
- ArchiveCustomField
-

ArchiveDamage

- ArchiveDeleteDamage

> **Note**
>
> OMS archive journals and real-time incident journals include several similar record types, but their content might be different. For example, ArchiveIncident records in a real-time incident journal show the initial states of incidents, while ArchiveIncident records in an OMS archive journal show the final states of incidents. For archiving, the latest record among the journals is used, such that the latest object state is recorded in the database.

The following record types are archived from the OMS archive journal only:

- ArchiveSwitchingOrder
- ArchiveSwitchingOrderCustom
- ArchiveSwitchingStepCustom

The following record types are archived from the real-time incident journal only:

- ArchiveAssessmentLocationUpdate
- ArchiveIncidentUpdate
- ArchiveRelatedIncidentIds
- ArchiveEtrUpdate
- ArchiveOrder
- ArchiveOrderUpdate
- ArchiveOrderDelete
- ArchivePlannedOutage
- ArchivePlannedOutageUpdate
- ArchivePlannedOutagePremises
- ArchivePlannedOutagePremisesUpdate
- ArchivePlannedOutagePremisesDelete
- ArchiveAreaNote
- ArchiveAreaNoteUpdate
- ArchiveAreaNoteChronology
- ArchiveCrewActivity
- ArchiveCrew
- ArchiveCrewUpdate
- ArchiveCrewDelete
- ArchiveIncidentNotActive

- ArchiveCallUpdate
- ArchiveCrewModelUpdate
- ArchiveEmployee
- ArchiveEmployeeUpdate
- ArchiveEmployeeDelete
- ArchiveVehicle
- ArchiveVehicleUpdate
- ArchiveVehicleDelete
- ArchiveVehicleType
- ArchiveVehicleTypeUpdate
- ArchiveVehicleTypeDelete
- ArchiveEmployeeTypeUpdate
- ArchiveEmployeeTypeDelete
- ArchiveEmployeeType
- ArchivePlannedOutageCustomField
- ArchiveCrewCustomField
- ArchiveAreaNoteCustomField
- ArchiveDamageUpdate

When already archived incidents are merged or split in History Correction, the data in the Archive database is updated accordingly. The following journal record types pertain only to incident merging or splitting in History Correction, and result in rows being deleted from OMS tables.

- ArchiveIncidentDelete
- ArchiveRestorationStepDelete
- ArchiveSwitchingStepDelete

Archived outage management data is stored in the INCIDENT, INCIDENT_CHRONOLOGY, AFFECTED_PREMISES, RESTORATION_STEP, SWITCHING_STEP_INCIDENT, DAMAGE_ASSESSMENT, CREW, CREW_ACTIVITY, EMPLOYEE, VEHICLE, FOLLOW_UP_ORDER, ITR_ETR, PLANNED_OUTAGE, AREA_NOTE, AREA_NOTE_CUSTOM, and other tables.

2.2.7. Switching Orders

Switching orders in Executed and Completed states are written to the OMS archive journal and stored in the Archive database in a similar manner as incidents.

The following journal record types are archived:

- ArchiveSwitchingStep

- ArchiveSwitchingStepCustom
- ArchiveSwitchingStepDelete
- ArchiveSwitchingOrder
- ArchiveSwitchingOrderCustom

Archived switching orders are stored in the SWITCHING_ORDER and SWITCHING_STEP_ORDER tables.

2.2.8. Feeder Updates

Feeder updates (for example, changes to the total load on a feeder) are written to the OMS archive journal and stored in the Archive database in a similar manner as incidents.

The following journal record types are archived:

- ArchiveFeederUpdate

Archived feeder updates are stored in the FEEDER table.

2.2.9. Incident Time Metrics

Incident time metrics (for example, time for an incident to change from the created state to the assigned state) are recorded in the Archive database based on real-time incident journal records.

Archived incident time metrics are stored in the INCIDENT_TIME_METRIC_EVENT table.

2.2.10. Calls, Meter Events, and Meter Statuses

Customer and unassociated calls received by Call Server, whether from the ADMS Call Entry application or from an external customer service system, are written by Call Server to a call journal. Similarly, meter events and meter statuses (sent in response to meter pings) are received by Call Server from external meter data applications and written to the call journal. This call journal data is used to initialize Call Server upon startup. It also is a data source for the Archive application.

The following call journal record types are archived:

- CallMessage
- MeterEvent
- MeterVoltEvent
- MeterStatus
- MeterVoltStatus

In addition, the following real-time incident record type is archived:

- ArchiveCallUpdate

Archived call and meter data are stored in the following tables, which correspond to the call journal record types listed above.

- CALL
- METER_EVENT
- METER_VOLTAGE_EVENT
- METER_STATUS
- METER_VOLTAGE_STATUS

Note that `MeterVoltStatus` records, and hence the corresponding METER_VOLTAGE_STATUS table, contain active and reactive power data as well as voltage data.

> **Note**
>
> Although call and real-time incident journals may contain similar data, some data may only be archived from a specific journal. For example, the REFERENCE ID value in the CALL table is archived from the call journal only.

2.2.11. Safety Documents

Released safety documents are written to archive files by the Safety Document application. If the safety document archive folder is on a different server than the Archive application, the safety document archive files may be transferred to the Archive application server by the File Updater application. The Archive application reads the safety document archive files and stores the data in the Archive database. After a safety document is archived, it is moved to a separate folder.

Archived safety documents are stored in the SAFETY_DOCUMENT, SAFETY_DOCUMENT_CREW, and SAFETY_DOCUMENT_DEVICE tables.

2.2.12. Tags

Tags are archived by the Historian application, which is separate from the ADMS Archive application. Historian is designed specifically for archiving data from Habitat applications. For more information, refer to the *Historian Analyst Guide*.

Although ADMS tags are not archived by ADMS Archive, they can be accessed from the ADMS Archive database via a database link to the Historian database. This enables report writers to access all ADMS data from a single logical data source.

2.2.13. Devices

Devices referenced by archived historical data (for example, incidents, switching orders, or safety

documents) are also archived. Other devices are not archived, even though they may exist in the Distribution Network Object Model (DNOM) or SCADA model (SCADAMOM).

When a record is archived, the referenced device is inserted or updated in the DEVICE table, as appropriate. The data for the new device record is derived from the object (for example, incident) data record. For example, when an incident is archived, if a device referenced by the incident does not exist in the DEVICE table, a device record is inserted.

Similar to devices, feeders that are associated with devices are archived in the FEEDER table.

Devices that are modeled in DNOM are identified by their DNOM device ID. Devices that are modeled in SCADA only, but not in DNOM, are identified by their SCADA composite IDs. SCADA composite IDs include four components (SUBSTN, DEVTYP, DEVICE, and POINT); therefore, it is possible for two points of a single SCADA-only device to appear in the Archive database as two separate devices.

Archived devices are stored in the DEVICE, DEVICE_FEEDER, DEVICE_SERV_CENTER, and DEVICE_STATION tables. The DEVICE_FEEDER, DEVICE_SERV_CENTER and DEVICE_STATION tables store the associations between devices and feeders, between devices and service centers, or between devices and stations, with start and end dates to track changes in the associations over time.

2.2.14. Enterprise OMS Data

Data from the Enterprise OMS system (for example, orders, tasks, or crew assignments) can be archived.

Archived Enterprise OMS data is stored in the ORDERS, TASKS, TASK_TYPE, DEVICE_OPERATIONS, and other tables.

2.2.15. Customer Data

You can archive customer data from a customer model file in the CUSTOMER table in the Archive database using the Customer Loader tool. For more information, refer to the *Customer List Compiler User Guide*.

> **Note**
>
> The Archive database performance increases when all customer data is archived in the CUSTOMER table, because in this case the Archive application does not need to archive customer information when a customer event appears (for example, a call or an incident).

The table below shows how columns map among fields in the customer schema file's output schema and the CUSTOMER table.

Table 1. Mapping between Customer Schema File and Customer Table

Schema Output Field Name	CUSTOMER Column Name	Comment
AccountNumber	ACCOUNT_NUMBER	
Address	ADDRESS	
AMI	AMI	
AmiLoad	AMI_LOAD	
AmiVolt	AMI_VOLT	
ApartmentNumber	APARTMENT_NUMBER	
AreaCode	AREA_CODE	
CriticalAccount	CRITICAL_ACCOUNT	
CriticalAccount2	CRITICAL_ACCOUNT_2	
CriticalAccount3	CRITICAL_ACCOUNT_3	
CriticalAccount4	CRITICAL_ACCOUNT_4	
CriticalAccount5	CRITICAL_ACCOUNT_5	
CustomerName	CUSTOMER_NAME	
CustomerTypeId	CUSTOMER_TYPE_ID	
City	ORG_LEVEL_2	
DerId	DER_ID	
Feeder	FEEDER	
FirstName	FIRST_NAME	
HazardousCustomer	HAZARDOUS_CUSTOMER	
InactiveAccount	INACTIVE_ACCOUNT	
LastName	LAST_NAME	
LoadId	LOAD_ID	
MeterNumber	METER_NUMBER	
MiddleName	MIDDLE_NAME	
PhoneNumber	PHONE_NUMBER	
PhonePrefix	PHONE_PREFIX	
PhoneSuffix	PHONE_SUFFIX	
PremiseId	PREMISE_ID	
ServiceCenter	SERVICE_CENTER	
ServicePoint	SERVICE_POINT	
SecondSevicePoint	SECOND_SERVICE_POINT	
SpecialAccount	SPECIAL_ACCOUNT	
StreetName	STREET_NAME	
StreetNumber	STREET_NUMBER	
StreetSuffix	STREET_SUFFIX	
X	X	

Schema Output Field Name	CUSTOMER Column Name	Comment
Y	Y	
Zipcode	ZIPCODE	
N/A	CUSTOMER_KEY	Composite field for a unique customer key.
N/A	IS_DELETED	Specifies whether a customer entry was deleted.
N/A	DELETED_TIME	Date and time when a customer entry was deleted.

2.2.16. Coded Translation Data

The `CodedTranslationConfig.xml` file includes configurations for mapping numeric values from code to human-readable descriptions, so that reports refer to meaningful descriptions instead of meaningless IDs or codes. For example, an incident may show the device type code of "16", but the report may show the device type description of "Distribution Transformer" instead.

The `CodedTranslationConfig.xml` file can be viewed and edited with the Coded Translation Editor. When the CodedTranslationConfig.xml file is updated in the directory monitored by the Archive application, this reference data is automatically updated in the Archive database as follows:

- Removed coded translation item parameters are still stored in the database. This is for consistency, because they may be referenced by other archive fields.
- New coded translation item parameters are added to the database.
- Updated coded translation item parameters (for example, with changed description) are updated in the database.

> **Note**
>
> The coded translation item parameter key is used to uniquely identify an item parameter in the Archive database.

The coded translation editor items that are stored in the Archive database are listed below, along with their associated database tables.

Item	Database Table
Material	MATERIAL
FailedEquipment	FAILED_EQUIPMENT
Reason	INCIDENT_CAUSE
DeviceType	DEVICE_TYPE
SwitchType	SWITCH_TYPE
SystemMode	SYSTEM_MODE
OutageIncidentType	INCIDENT_TYPE

Item	Database Table
OutageType	OUTAGE_TYPE
SwitchOrderType	SWITCHING_ORDER_TYPE
SwitchOrderWorkType	SWITCHING_ORDER_WORK_TYPE
ActionTaken	INCIDENT_ACTION
PowerQualityType	POWER_QUALITY_TYPE
NonoutageType	NONOUTAGE_TYPE
SystemAffected	SYSTEM_AFFECTED
Weather	WEATHER
WorkQueues	WORK_QUEUES
FailureMode	FAILURE_MODE
Manufacturer	MANUFACTURER
Animals	ANIMALS
IncidentEventType	INCIDENT_EVENT_TYPE
AreaNote	AREA_NOTE_CASE_NOTE
PlannedOutageType	PLANNED_OUTAGE_TYPE
PlannedOutageStatus	PLANNED_OUTAGE_STATUS
IncidentWorkingStatus	INCIDENT_STATUS
Plants	PLANTS
Condition_1	CALL_CONDITION1
Condition_2	CALL_CONDITION2
Description	CALL_DESCRIPTION
Priority_Code	CALL_PRIORITY
CallType	CALL_TYPE
OrderStatus	WORK_ORDER_STATUS
OrderOrganization	WORK_ORDER_ORGANIZATION
OrderType	WORK_ORDER_TYPE
VoltageEventType	METER_VOLTAGE_EVENT_TYPE
FaultLocationSource	FAULTLOCATIONSOURCE
TroubleDescription	TROUBLE_DESCRIPTION
ReasonGroup	INCIDENT_CAUSE_GROUP
OutageDeviceType	OUTAGE_DEVICE_TYPE
ARMSStatus	CREW_STATUS
CustomerType	CUSTOMER_TYPE
DeviceClass	DEVICE_CLASS
SuppressReason	SUPPRESS_REASON
CaseNote	CASE_NOTE

2.2.17. Company-Specific Data

Company-specific data does not exist in ADMS, so it cannot be loaded by the Archive application.

For the system to function correctly, or to enable certain types of reports to be produced, the following tables or columns are loaded and maintained manually (not by the Archive application):

- PD_AREA
- PD_OPERATION
- ORG_HIERARCHY_TYPE
- ORG_UNIT_TYPE
- ORG_UNIT_HIERARCHY: Populated using the `ORG_UNIT_HIERARCHY.sql` script generated by the Customer List Compiler tool.
- ORG_UNIT: Populated using the `ORG_UNIT.sql` script generated by the Customer List Compiler tool.
- SERVICE_CENTER: Populated using the `SERVICE_CENTER.sql` script generated by the Customer List Compiler tool.
- SERVICE_CENTER_ORG: Populated using the `SERVICE_CENTER_ORG.sql` script generated by the Customer List Compiler tool.
- STATION
- FEEDER
- MAJOR_EVENT
- Incident.Event_Id: Foreign key to Major_Event.Event_Id.
- CUSTOMER: Created using an SQL script generated by the Customer List Compiler tool. Populated using the Customer Loader application.

> **Note**
>
> "PD" in PD_AREA and PD_OPERATION stands for "Power Delivery".

The following tables need to be populated to enable the Major Event Day (MED) functionality:

- MED_PARAMETERS
- MED_DAILY_SAIDI
- MED_SERV_CENTER_BMEE
- MED_BMEE_CUSTOMER_COUNT
- MED_EXCLD_INCIDENT_CAUSE
- MED_EXCLD_OUTAGE_TYPE
- MED_EXCLD_DEVICE_TYPE
- MED_BMEE

For more information about the Major Event Day (MED) functionality, see Major Event Day Calculations.

[1] LVM is sometimes referred to as Volt/VAr Control (VVC).

3. Configuring the Archive Application

Archive application configuration parameters are specified in configuration file `ArchiveAppConfig.xml`. This file is installed in the same folder as other ADMS server configuration files (for example, %IDMS_ROOT%\config\server). A custom editor is not provided for this configuration file; it must be edited via a text editor such as Notepad++.

3.1. Configuration Parameters

The configuration file includes comments with brief explanations of each configuration parameter. This section provides detailed explanations of each configuration parameter.

3.1.1. DataProviderName

The invariant name of the ADO.NET data provider used for data access. Theinvariant name of the Oracle Data Provider for .NET (ODP.NET) Managed Driver is `Oracle.ManagedDataAccess.Client`. Theinvariant name of the ODP.NET Unmanaged Driver is `Oracle.DataAccess.Client`. The invariant name of .NET Framework Data Provider for SQL Server (SqlClient) is `System.Data.SqlClient`. For other data providers, refer to the data provider's documentation. A list of ADO.NET data providers is available at http://msdn.microsoft.com/en-us/data/dd363565. ADMS Archive is certified for use only with ODP.NET (Managed Driver or Unmanaged Driver) or SqlClient.

> **Note**
>
> ODP.NET 12.2c Release 1 includes both a Managed Driver and an Unmanaged Driver. Although ADMS Archive works with either driver, it is configured for the Managed Driver. The default configuration could be overridden via an application configuration file (`ArchiveApp.exe.config`), but since the Unmanaged Driver provides no functionality required by Archive that is not provided by the Managed Driver, an `ArchiveApp.exe.config` file is not supplied in ADMS releases. For an Oracle database, set DataProviderName to `Oracle.ManagedDataAccess.Client`.

The Entity Framework metadata files, `Archive_Oracle.ssdl` and `Archive_SqlServer.ssdl`, also specify the data provider name in the Provider attribute of the Schema element. The data provider name specified in the SSDL file must be the same as the data provider name specified by the DataProviderName configuration parameter.

- In Archive_Oracle.ssdl, if you are using ODP.NET with the Managed Driver, set Provider="Oracle.ManagedDataAccess.Client".
- If you are using ODP.NET with the Unmanaged Driver, set Provider="Oracle.DataAccess.Client".
- In Archive_SqlServer.ssdl, Provider="System.Data.SqlClient", which is correct for all versions of SqlClient.

3.1.2. DbConnectionString

This configuration parameter contains three attributes: "value", "encrypted", and "keyContainerName". The "value" attribute specifies the database connection string. The "encrypted" attribute specifies whether the connection string is encrypted. The "keyContainerName" attribute specifies the name of the RSA encryption key, if applicable (see below). Unlike other configuration parameters, the connection string is specified as an XML attribute value rather than an XML element value.

The Archive application uses the connection string to connect to the Archive database. The configuration file contains default connection strings for Oracle and SQL Server databases. The default connection string for an Oracle database is shown below:

```
Data Source=//host:port/service_name;
User Id=user_id;
Password=password;
Min Pool Size=0;
Validate Connection=true
```

Replace the italicized words above with valid values for your database. For example:

```
Data Source=//dbserver01:1521/IDMS;
User Id=ARCHIVE;
Password=ARCHIVE;
Min Pool Size=0;
Validate Connection=true
```

> **Note**
>
> Because of the line width limitations of this document, the connection strings above are shown on multiple lines. In the configuration file, the connection string should be on one line.

Connection strings are different for each data provider. Connection strings may contain many other attributes besides those that are specified in the default ArchiveAppConfig.xml. For details, refer to your data provider's documentation.

- For ODP.NET, refer to:

  Oracle Data Provider for .NET Developer's Guide
  http://docs.oracle.com/cd/E51173_01/win.122/e17732.pdf

  Oracle Data Provider for .NET / ODP.NET connection strings
  http://www.connectionstrings.com/oracle-data-provider-for-net-odp-net/

- For .NET Framework Data Provider for SQL Server, refer to:

  SqlConnection.ConnectionString Property
  http://msdn.microsoft.com/en-us/library/system.data.sqlclient.sqlconnection.connectionstring%28v=vs.110%29.aspx

NET Framework Data Provider for SQL Server connection strings

http://www.connectionstrings.com/sqlconnection/

You should be able to connect to the database with a database tool such as SQL*Plus using the database user ID, password, and data source that are specified in this connection string. If you cannot connect to the database, the Archive application may not be able to connect to the database. Database connection problems are common for a new installation, so be sure that `DbConnectionString` is set correctly. The database administrator who created the database should be able to provide the connection string.

3.1.2.1. Encrypting the Connection String

Specifying the database connection string in clear text may be a security risk, especially for a production database. Therefore, the connection string may be encrypted using the Encryption Tool. For more information, refer to the *ADMS Software Installation and Maintenance Guide*.

3.1.2.2. External Connection String Configuration File

If the database connection string is encrypted with a password-based Key, it may be desirable to specify the encrypted connection string in an external configuration file, because a connection string encrypted on one computer (for example, the primary server in a high availability configuration) cannot be decrypted on another computer (for example, the standby server in a high availability configuration). This will allow `ArchiveAppConfig.xml` to be copied from the primary server to the standby server, since it does not include an encrypted connection string. The external configuration file containing the encrypted connection string, however, cannot be copied from the primary server to the standby server since the encrypted connection string on the standby server will be different than the encrypted connection string on the primary server.

To specify the database connection string in an external configuration file, create a separate XML file containing only the DbConnectionString element, and specify that file's path in the configSource attribute of the DbConnectionString element in `ArchiveAppConfig.xml`. For example, the following entry in `ArchiveAppConfig.xml` references an external `ArchiveDbConnectionStringConfig.xml` file that specifies the database connection string:

```
<DbConnectionStringconfigSource="%IDMS_ROOT%\config\server\ArchiveDbConnectionStringConfig.xml" />
```

3.1.2.3. Connection Pooling

Given that opening and closing connections is relatively resource intensive, connection pooling improves performance by reusing connections. Connection pooling is enabled by default and may be controlled by specifying connection string attributes.

Most of the default values of the connection string attributes related to connection pooling are appropriate for the Archive application. However, for Oracle database connection pooling with

Oracle Data Provider for .NET (ODP.NET), GE recommends setting "Min Pool Size=0" and "Validate Connection=true" in order to override their default values.[1]

"Min Pool Size=0" allows the number of connections in the connection pool to be reduced to zero. This not only closes all connections during idle periods, but also prevents errors that frequently occur when a connection is open for a long time.

"Validate Connection=true" causes validation of connections obtained from the pool. Although this causes a round trip to the database for each pool connection, it prevents database connection errors that occur when a session is disconnected. Since the transaction volumes of the Archive application are relatively low, the extra overhead of validating connections is acceptable. Validating connections avoids the need to restart the Archive application, or to manually move safety document files from the SafetyDocumentsNotArchivedPath folder to the SafetyDocumentsToArchivePath folder, after connection errors occur.

3.1.2.4. Multiple Active Result Sets

SQL Server connections should enable Multiple Active Result Sets by specifying "MultipleActiveResultSets=true" in the connection string. The default SQL Server connection strings in `ArchiveAppConfig.xml` include this attribute. For more information about Multiple Active Result Sets, refer to the SQL Server documentation.

> **Note**
>
> You must add "MultipleActiveResultSets=true" in the connection string when using multiple instances of the Archive applications.

3.1.3. Metadata

This configuration parameter specifies the paths of three files that contain metadata for the Entity Data Model of the Archive database. The three files specify the Conceptual Schema Definition Layer (CSDL), Store Schema Definition Layer (SSDL), and Mapping Schema Layer (MSL) of the entity data model. The paths of all three files are configurable.

Since the SSDL specifies DBMS-specific data types (for example, nvarchar2 for Oracle or nvarchar for SQL Server), DBMS-specific SSDL files (`Archive_Oracle.ssdl` and `Archive_SqlServer.ssdl`) are supplied. The CSDL and MSL do not specify DBMS-specific data types. DBMS-specific CSDL files (`Archive_Oracle.csdl` and `Archive_SqlServer.csdl`) and MSL files (`Archive_Oracle.msl` and `Archive_SqlServer.msl`) are supplied to enable certain entities to be defined in only one of the two supported database management systems. This generally will occur only in prerelease development kits, when a new feature has been developed with one DBMS but not yet tested with the other DBMS. Typically, `Archive_Oracle.csdl` will be identical to `Archive_SqlServer.csdl` and `Archive_Oracle.msl` will be identical to `Archive_SqlServer.msl`.

The full configuration parameter value should therefore be one of the following, depending on whether your database is Oracle or SQL Server:

```
./Archive_Oracle.csdl|./Archive_Oracle.ssdl|./Archive_Oracle.msl
./Archive_SqlServer.csdl|./Archive_SqlServer.ssdl|./Archive_SqlServer.msl
```

In the paths above, "." refers to the directory containing the `ArchiveApp.exe` assembly. It may be replaced with another absolute or relative path name if the files are installed in another directory.

The SSDL files specified by the Metadata configuration parameter, `Archive_Oracle.ssdl` and `Archive_SqlServer.ssdl`, contain Provider and ProviderManifestToken attributes in the Schema element. The values of those attributes may need to be changed for your environment.

The Provider attribute specifies the data provider name. It must match the data provider name specified by the DataProviderName configuration parameter.

- In `Archive_Oracle.ssdl`, Provider="Oracle.ManagedDataAccess.Client". That is the invariant name for Oracle Data Provider for .NET with the Managed Driver.
- If you are using ODP.NET with the Unmanaged Driver, set Provider to "Oracle.DataAccess.Client".
- In `Archive_SqlServer.ssdl`, Provider="System.Data.SqlClient"; this Provider name does not need to be changed.

The ProviderManifestToken attribute specifies the target database engine release. SQL queries generated by the data provider for Entity Framework may vary depending on this value:

- In `Archive_Oracle.ssdl`, set ProviderManifestToken="12.1", which is appropriate for Oracle 12c databases. For Oracle 11g Release 2 databases, set ProviderManifestToken="11.2".
- In `Archive_SqlServer.ssdl`, set ProviderManifestToken="2008". This value is appropriate for SQL Server 2014 and SQL Server 2012, since .NET Framework 4.5 does not support ProviderManifestToken="2014" or "2012".

> **Note**
>
> ProviderManifestToken specifies the database compatibility level, not the release number of the data provider (ODP.NET or SqlClient). If you are using ODP.NET 12.2c Release 1 to connect to an Oracle 11g Release 2 database, ProviderManifestToken should be "11.2" rather than "12.1".
> Since many SQL queries generated for one database release may execute correctly on another database release, Archive may work correctly even if ProviderManifestToken is incorrect. But you should set it appropriately for your database to benefit from optimizations as well as to prevent the possibility of generating SQL that will not execute on your database server.

3.1.4. CustomSecretStore

The Archive application can retrieve the Archive database password stored in the CyberArk Enterprise Password Vault so that you do not need to provide the password in ArchiveAppConfig.xml (as a plain text or encrypted string).

For more information about storing passwords in CyberArk. refer to the *ADMS Software Installation and Maintenance Guide*.

The default configuration is as follows:

```
<CustomSecretStore>
  <AssemblyName>cyberArkSecret.dll</AssemblyName>
  <ClassName>CyberArkSecret.CyberArkSecretStore</ClassName>
  <ObjectKey>ETD,ETD_Databases,root,Application-CyberArkPTA-User-ARCHIVE-ARCHIVE</ObjectKey>
</CustomSecretStore>
```

Where:

- **AssemblyName:** Name of the assembly (DLL) containing a class that implements the CyberArk secret store.
- **ClassName:** Name of the class that implements the CyberArk secret store. This class is implemented based on the CyberArk software development kit (SDK) and supports connecting to CyberArk API.
- **ObjectKey:** CyberArk object key, which contains CyberArk application ID, safe, database (folder), and object definitions of the database password defined in the Enterprise Vault Server. The format is <DatabaseMemberUserName>,<AccountSafe>,root,Application-<AccountPlatformID>-<AccountAddress>-<AccountUsername>

> **Note**
>
> When you provide the password both in the database connection string and the Custom Secret Store node, the application first tries to retrieve the password from the connection string, and then from CyberArk.

3.1.5. UseDnomDb

Indicator of whether the Archive application should look up device data from the DNOM database when that device data is not provided in the Archive data records (for example, incidents). Archive data records that reference devices include several fields that describe the device, including the device's ID, name, type, switch type (if the device is a switch), nominal feeder, station, and operating center. In some cases, not all these fields are populated on the source data records because the values are not available from the source applications. In these cases, the Archive application may look up the "missing" values from the DNOM database.

This feature enables the device data in the Archive database to be more complete, thereby facilitating reporting. However, for the Archive application to look up device data from the DNOM database, the DNOM database must exist and be loaded with data from the distribution network (station) model files by the ADMS DNOM Database Loader application. If the DNOM database does not exist in your installation, performance of the Archive application will be improved slightly by setting this configuration parameter to "false", which indicates to the Archive application that it should not attempt to lookup device data from the DNOM database. This is particularly relevant to

installations that do not archive OMS data (incidents and switching orders) or safety documents.

For more information about loading the DNOM database, see DNOM Database Loader.

3.1.6. DnomDbFunctionNameQualifier

If UseDnomDb is True, the Archive application calls stored functions in the DNOM database to look up device data that is not provided in the Archive data records (for example, incidents). Although the function names are known, the fully qualified names depend upon the DNOM database installation. This configuration parameter specifies a qualifier that is prepended to the function names in order to create fully qualified function names that enable the DNOM database functions to be called by the Archive application.

In Oracle, the DNOM database functions are defined in a package named DNOM_DB. A public synonym for that package, also named DNOM_DB, is created. Thus, the Archive application may reference a DNOM database function by prefixing the function name with "DNOM_DB" (for example, `DNOM_DB.GetValue`). Therefore, for an Oracle database, you should set DnomDbFunctionNameQualifier to "DNOM_DB". Alternatively, if you prefer not to use the DNOM_DB public synonym, you may set `DnomDbFunctionNameQualifier` to the DNOM database schema name (for example, "DNOM").

SQL Server does not have packages. Consequently, in SQL Server, fully qualified function names include the database name and schema name as well as the function name (for example, `DNOM.dbo.GetValue`). Therefore, for a SQL Server database, `DnomDbFunctionNameQualifier` should be set to the DNOM database name and schema name, separated by a period (for example, "DNOM.dbo").

3.1.7. ShutdownDelaySeconds

Number of seconds to delay shutdowns (and hence automatic restart attempts by PROCMAN). When Archive shuts down, PROCMAN attempts to restart it immediately. If the shutdown is due to a temporary database connection failure (for example, a failover), this can result in many unsuccessful restart attempts (a future PROCMAN version will include a configuration parameter to delay restart attempts). Archive provides the ShutdownDelaySeconds configuration parameter to delay shutdowns, which is an indirect way to delay restarts by PROCMAN. This configuration parameter may be deleted when the new PROCMAN version is implemented, or when Connection Resiliency is implemented. Connection Resiliency is a feature of Entity Framework 6.0 that is not yet supported by Oracle Data Provider for .NET (ODP.NET), as of September 2015.

3.1.8. ArchiveReferenceData

Indicator of whether the Archive application should archive reference data, which includes static codes and descriptions such as device types and outage types. If this configuration parameter is set to "false", reference data is not archived, and the corresponding Threads menu item and Log

Message tab page are removed. The parameter should only be set to "false" in a configuration that includes more than one instance of the Archive application, and another Archive application instance has this configuration parameter set to "true".

3.1.9. ArchiveReferenceDataLogFilePathBase

Path base (excluding file name date and extension) to the log file for archived reference data. Log file paths are this base value concatenated with the date and file extension. The default date format is **yyyy-MM-dd** and the default log file extension is **.log** but both are localizable. For example, if the path base is "D:\log\ArchiveReferenceData", the log file path for July 31, 2018 is D:\log\ArchiveReferenceData_2018-07-31.log.

3.1.10. ReferenceDataFilePath

Path to the folder that contains the `CodedTranslationConfig.xml` file. The Archive application imports some reference data from that file into the database.

3.1.11. ArchiveDmsData

Indicates whether the Archive application should archive DMS data. If this configuration parameter is set to "false", DMS data is not archived and the corresponding Threads menu item and Log Message tab page are removed.

3.1.12. ArchiveDmsDataLogFilePathBase

Path base (excluding file name date and extension) to the log file for archived DMS data. Log file paths are this base value concatenated with the date and file extension. The default date format is **yyyy-MM-dd** and the default log file extension is **.log** but both are localizable. For example, if the path base is "D:\log\ArchiveDmsData", the log file path for July 31, 2018 is D:\log\ArchiveDmsData_2018-07-31.log.

3.1.13. DmsDataPath

Path to the directory containing DMS archive journal files. The Archive application queries these DMS archive journal files for new data to archive.

3.1.14. DmsDataArchivedPath

Path to the directory containing old (retired) DMS archive journal files whose data has already been stored in the archive database. The Archive application moves files from DmsDataPath to

DmsDataArchivedPath a day after information from a journal has been archived to minimize the number of DMS archive journal files that must be read in queries for new data to archive.

> **Note**
>
> On standby servers, DMS archive journal files are not retired automatically because the Archive application retires journal files only on the server where it is running.

3.1.15. DmsDataPollingIntervalSeconds

Number of seconds between queries for new DMS data to archive. This should be a small number so that data is archived shortly after it is loaded into the DMS server.

> **Note**
>
> The Archive application uses FileSystemWatcher (a .NET Framework class) to detect new safety document archive files or changes to the reference data file. However, the Archive application uses polling to detect changes to journal files because journal file changes are not detected by FileSystemWatcher.

3.1.16. DmsDataLastJournalTimeArchivedFileName

Name of the file (in the folder specified by DmsDataPath) that contains the journal time of the last DMS archive journal entry that has been stored in the Archive database.

A file management tool may periodically delete DMS archive journal files from the folder that is specified by DmsDataPath so that the folder does not grow indefinitely and run out of disk space. The file management tool may delete old files, but should leave newer files that contain data that has not been archived to the database (especially if the Archive application is down for an extended period). The file specified by this configuration parameter specifies the time after which DMS archive journal files should not be deleted by the file management tool.

3.1.17. ArchiveOmsData

Indicates whether the Archive application should archive OMS data. In this context, "OMS data" refers to data written by OMS Server (Net Server) to the OMS archive journal. It includes switching orders as well as incidents, even though switching orders are not OMS data, because switching orders are written to the OMS archive journal. It excludes calls and meter data, which logically are OMS data, because they are not written to the OMS archive journal. If this configuration parameter is set to "false", OMS data (incidents and switching orders) is not archived and the corresponding Threads menu item and Log Message tab page are removed.

OMS Server writes both incidents and switching orders to the OMS archive journal. It cannot be configured to only write certain types of archive journal entries. Similarly, this configuration

parameter controls archiving of all record types in the OMS archive journal. Thus, it is not possible to archive, for example, switching orders but not incidents.

3.1.18. ArchiveOmsDataLogFilePathBase

Path base (excluding file name date and extension) to the log file for archived OMS data (incidents and switching orders). Log file paths are this base value concatenated with the date and file extension. The default date format is **yyyy-MM-dd** and the default log file extension is **.log** but both are localizable. For example, if the path base is "D:\log\ArchiveOmsData", the log file path for July 31, 2018 is D:\log\ArchiveOmsData_2018-07-31.log.

3.1.19. OmsDataPath

Path to the directory containing OMS archive journal files. The Archive application queries these OMS archive journal files for new incidents and switching orders to archive.

3.1.20. OmsDataArchivedPath

Path to the directory containing old (retired) OMS archive journal files whose data has already been stored in the archive database. The Archive application moves files from OmsDataPath to OmsDataArchivedPath a day after information from a journal has been archived to minimize the number of OMS archive journal files that must be read in queries for new data to archive.

> **Note**
>
> On standby servers, OMS archive journal files are not retired automatically because the Archive application retires journal files only on the server where it is running.

3.1.21. OmsDataPollingIntervalSeconds

Number of seconds between queries for new OMS data to archive. This should be a small number so that incidents and switching orders are archived shortly after they are cleared (incidents) or executed (switching orders).

> **Note**
>
> The Archive application uses FileSystemWatcher (a .NET Framework class) to detect new safety document archive files or changes to the reference data file. However, the Archive application uses polling to detect changes to journal files because journal file changes are not detected by FileSystemWatcher.

3.1.22. OmsDataLastJournalTimeArchivedFileName

Name of the file (in the folder specified by OmsDataPath) that contains the timestamp of the last OMS archive journal entry that has been stored in the Archive database.

A file management tool may periodically delete OMS archive journal files from the folder that is specified by OmsDataPath so that the folder does not grow indefinitely and run out of disk space. The file management tool may delete old files, but should leave newer files that contain data that has not been archived to the database (especially if the Archive application is down for an extended period). The file specified by this configuration parameter specifies the time after which OMS archive journal files should not be deleted by the file management tool.

3.1.23. ArchiveSwitchingOrders

Indicates whether the Archive application should archive switching order data from the OMS archive journal.

3.1.24. ArchiveNrtData

Indicates whether the Archive application should archive Near Real-Time (NRT) data. In this context, "NRT data" refers to near real-time outage management data written by Net Data Server or OMS Server (NetServer) to the real-time incident journal. If this configuration parameter is set to false, NRT data is not archived and the corresponding Threads menu item and Log Message tab page are removed.

> **Note**
>
> To enable archiving NRT data, creating real-time incident journals by Net Data Server must be enabled. For more information, refer to the *Outage Management Read Only Interface Programmer Guide*.

> **Note**
>
> If the ADMS Near Real-Time Database is used as the source for outage maps, the outage map queries should be run on a replica of the Near Real-Time Database. This avoids delays in inserting or updating data in the primary Near Real-Time Database.

3.1.25. ArchiveNRTIncident

Indicates whether the Archive application should archive incident data from the real-time incident journal.

> **Note**

For archiving NRT data, the ArchiveNrtData parameter must be set to True.

3.1.26. ArchiveNRTWorkOrder

Indicates whether the Archive application should archive follow-up work order data from the real-time incident journal.

3.1.27. ArchiveNRTDamageAssessment

Indicates whether the Archive application should archive damage assessment location data from the real-time incident journal.

3.1.28. ArchiveNRTAreaNote

Indicates whether the Archive application should archive area note data from the real-time incident journal.

3.1.29. ArchiveNRTCrew

Indicates whether the Archive application should archive crew data from the real-time incident journal.

3.1.30. ArchiveNRTCall

Indicates whether the Archive application should archive call data from the real-time incident journal.

3.1.31. ArchiveNRTEmployee

Indicates whether the Archive application should archive employee data from the real-time incident journal.

3.1.32. ArchiveNRTPlannedOutage

Indicates whether the Archive application should archive planned outage data from the real-time incident journal.

3.1.33. ArchiveNRTVehicle

Indicates whether the Archive application should archive vehicle data from the real-time incident journal.

3.1.34. ArchiveNrtDataLogFilePathBase

Base path (excluding file name date and extension) to the log file for archived NRT data (incidents and switching orders). Log file paths are this base value concatenated with the date and file extension. The default date format is **yyyy-MM-dd** and the default log file extension is **.log**, but both are localizable. For example, if the base path is D:\log\ArchiveNrtData, the log file path for July 31, 2018 is D:\log\ArchiveNrtData_2018-07-31.log.

3.1.35. NrtDataPath

Path to the directory containing real-time incident journal files. The Archive application queries these real-time incident journal files for near real-time outage management data to archive.

3.1.36. NrtDataArchivedPath

Path to the directory containing old real-time incident journal files whose data has already been stored in the archive database. The Archive application moves files from NrtDataPath to NrtDataArchivedPath to minimize the number of real-time incident journal files that must be read in queries for new data to archive.

3.1.37. NrtDataPollingIntervalSeconds

Number of seconds between queries for new NRT data to archive. This should be a small number so that the data is archived shortly after the journals are updated.

> **Note**
>
> The Archive application uses FileSystemWatcher (a .NET Framework class) to detect new safety document archive files or changes to the reference data file. However, the Archive application uses polling to detect changes to journal files because journal file changes are not detected by FileSystemWatcher.

3.1.38. NrtDataLastJournalTimeArchivedFileName

Name of the file (in the folder specified by NrtDataPath) that contains the timestamp of the last real-time incident journal entry that has been stored in the Archive database.

A file management tool may periodically delete real-time incident journal files from the folder that is specified by NrtDataPath so that the folder does not grow indefinitely and run out of disk space. The file management tool may delete old files, but should leave newer files that contain data that has not been archived to the database (especially if the Archive application is down for an extended period). The file specified by this configuration parameter specifies the time after which real-time incident journal files should not be deleted by the file management tool.

3.1.39. NrtIncidentTimeMetricEvents

This node includes configurations for events, for which time metrics are recorded in the ARCHIVE.INCIDENT_TIME_METRIC_EVENT table. For example, the system can record time for an incident to move from the created state to the assigned state.

By default, the following event metrics are defined and recorded:

- IncidentCreated
- CrewRequested
- CrewAssigned
- CrewOnSite
- WorkComplete
- PendingClear
- PowerRestored
- IncidentCleared

For example, a parameter value for the WorkComplete event is as follows:

```
<NrtIncidentTimeMetricEvent eventName="5" journalName="ArchiveIncidentUpdate" fieldName="ORDERSTATUS"
newValue="7" />
```

Where:

- eventName: The event to monitor. Events are defined using the IncidentEventType coded translation entry. The eventName value must be set to the associated IncidentEventType entries' key. For example, the WorkComplete key is "5".
- journalName: The name of the journal type that is expected.
- fieldname: The name of the field in the journal type that is expected (optional).
- newValue: The value of the field that is expected (optional).
- enabled: Specifies whether collecting metrics for the event is enabled (optional, "True" by default).

3.1.40. ArchiveWorkModelData

Note

This feature requires integration with the Enterprise OMS module.

The ArchiveWorkModelData feature indicates whether the Archive application should archive Enterprise OMS data. In this context, "Enterprise OMS data" refers to data written by Work Order Journal Adapter to the Work Model journal. If this configuration parameter is set to false, Enterprise OMS data is not archived and the corresponding Threads menu item and Log Message tab page are removed.

Note

To enable archiving Enterprise OMS data, creating Work Model journals by Work Order Journal Adapter must be enabled. For more information, refer to the *Enterprise OMS Software Installation and Maintenance Guide*.

3.1.41. ArchiveWorkModelDataLogFilePathBase

Path base (excluding file name date and extension) to the log file for archived Enterprise OMS data (orders and tasks). Log file paths are the base value concatenated with the date and file extension. The default date format is "yyyy-MM-dd" and the default log file extension is ".log", but both are localizable.

For example, if the path base is "D:\log\ArchiveWorkModelData", the log file path for July 31, 2018 is "D:\log\ArchiveModelsData_2020-05-22.log".

3.1.42. WorkModelDataPath

Path to the directory containing Work Model journal files. The Archive application queries these Work Model journal files for new data to archive.

3.1.43. WorkModelDataArchivedPath

Path to the directory containing old Work Model journal files whose data has already been stored in the archive database. The Archive application moves files from WorkModelDataPath to WorkModelDataArchivedPath to minimize the number of Work Model journal files that must be read in queries for new data to archive.

3.1.44. WorkModelDataPollingIntervalSeconds

Number of seconds between queries for new Enterprise OMS data to archive. This should be a small number so that the data is archived shortly after the journals are updated.

Note

> The Archive application uses FileSystemWatcher (a .NET Framework class) to detect new safety document archive files or changes to the reference data file. However, the Archive application uses polling to detect changes to journal files because journal file changes are not detected by FileSystemWatcher.

3.1.45. WorkModelDataLastJournalTimeArchivedFileName

Name of the file (in the folder specified by WorkModelDataPath) that contains the timestamp of the last Work Model journal entry that has been stored in the Archive database.

A file management tool may periodically delete Work Model journal files from the folder that is specified by WorkModelDataPath so that the folder does not grow indefinitely and run out of disk space. The file management tool may delete old files, but should leave newer files that contain data that has not been archived to the database (especially if the Archive application is down for an extended period). The file specified by this configuration parameter specifies the time after which Work Model journal files should not be deleted by the file management tool.

3.1.46. ArchiveCallJournalDataTypes

Indicates which Call Server call journal data types to archive. This configuration parameter contains one attribute per call journal record type. If all of them are set to "false", call journal data is not archived, and the corresponding Threads menu item and Log Message tab page are removed.

Note that with reference data, DMS data, OMS data, and safety documents, the corresponding Archive configuration parameter contains a single true or false value, which controls whether the Archive application archives data of that category. In each of those cases, Archive either archives all data from the data source, or none. (DMS Interface Server may be configured to write only certain data types to the DMS archive journal, though.) Since call journal data is used by Call Server as well as by Archive, the call journal may contain record types that need not be archived. Therefore, this configuration parameter allows you to specify which record types you want to archive. You may, for example, choose to archive calls, meter events, and meter voltage events, but not meter statuses and meter voltage statuses.

3.1.47. ArchiveCallJournalDataLogFilePathBase

Path base (excluding file name date and extension) to the log file for archived safety documents. Log file paths are this base value concatenated with the date and file extension.

The default date format is **yyyy-MM-dd** and the default log file extension is **.log** but both can be localized. For example, if the path base is "D:\log\ArchiveCallJournalData", the log file path for July 31, 2018 is D:\log\ArchiveCallJournalData_2018-07-31.log.

3.1.48. CallJournalFileName

The name of call journal files in the directory that contains call journal files. The Archive application queries call journal files for new calls and meter data to archive.

3.1.49. CallJournalDataPath

Path to the directory that contains call journal files. The Archive application queries these call journal files for new calls and meter data to archive.

> **Note**
>
> Both Call Server and OMS Server (Net Server) write a call journal. This configuration parameter should point to the Net Server's call journal. However, archiving the Call Server's call journal is also supported for backward compatibility.

3.1.50. CallJournalDataPollingIntervalSeconds

Number of seconds between queries for new call journal data to archive. This should be a small number so that calls and meter data are archived shortly after they are received.

> **Note**
>
> The Archive application uses FileSystemWatcher (a .NET Framework class) to detect new safety document archive files or changes to the reference data file. However, the Archive application uses polling to detect changes to journal files because journal file changes are not detected by FileSystemWatcher.

3.1.51. ArchiveSafetyDocuments

Indicates whether the Archive application should archive safety documents. If this configuration parameter is set to "false", safety documents are not archived, and the corresponding Threads menu item and Log Message tab page are removed.

3.1.52. ArchiveSafetyDocumentsLogFilePathBase

Path base (excluding file name date and extension) to the log file for archived safety documents. Log file paths are this base value concatenated with the date and file extension. The default date format is **yyyy-MM-dd** and the default log file extension is **.log** but both are localizable. For example, if the path base is "D:\log\ArchiveSafetyDocuments", the log file path for July 31, 2018 is D:\log\ArchiveSafetyDocuments_2018-07-31.log.

3.1.53. SafetyDocumentsToArchivePath

Path to the folder that contains safety documents to be archived. Released safety documents are written to this folder either directly by the Safety Document application or by the File Updater application.

3.1.54. SafetyDocumentsArchivedPath

Path to the folder that contains safety documents that have been archived. When the Archive application has successfully archived a safety document, it moves the safety document file to this folder so that the safety document is not re-archived.

3.1.55. SafetyDocumentsNotArchivedPath

Path to the folder that contains safety documents that have not been archived because of errors. If an error prevents the Archive application from archiving a safety document, the safety document is moved to this folder so that it can be re-archived (by moving the file to the path specified by SafetyDocumentsToArchivePath) once the error is corrected.

3.1.56. CoordinateConverterAssembly and CoordinateConverterTypeName

The CoordinateConverterAssembly define the assembly that is used to convert Projected Coordinate System (PCS) coordinates to system coordinates and vice versa. The CoordinateConverterTypeName define the class that is used to convert Projected Coordinate System (PCS) coordinates to system coordinates and vice versa. Coordinate conversion is required to show the latitude, longitude, x, and y values for customer calls in the CALL table.

- **CoordinateConverterAssembly:** Name of the assembly (DLL) containing a class that converts latitude/longitude to system X/Y coordinates.
- **CoordinateConverterTypeName:** Name of the class that converts between latitude/longitude to system X/Y coordinates.

For US customers, the default coordinate converter can be used. For non-US customers, a custom converter must be developed.

[1] These attributes are valid for connection strings in Oracle Data Provider for .NET (ODP.NET), but not for connection strings in Microsoft .NET Framework Data Provider for SQL Server.

4. Running the Archive Application

This chapter describes how to start the Archive application and describes the messages that the Archive application generates and logs.

4.1. Starting the Archive Application

To start the Archive application:

1. From the Windows Start menu, navigate to All Programs > Eterra > Distribution > Server > Archive.
2. Execute the following command from the folder that contains ADMSServer assemblies (for example, %IDMS_ROOT%\server\bin):

   ```
   ArchiveApp.exe/CONFIG<Full_Path_to_ArchiveAppConfig.xml>
   ```

   This command can be included in the ADMSProcess Starter to automatically start the Archive application along with other ADMSapplications.

4.2. Purging the Archive Database

To purge all incidents that were last updated before a specific date:

- Execute the following command from the folder that contains ADMSServer assemblies (for example, %IDMS_ROOT%\server\bin):

  ```
  ArchiveApp.exe /CONFIG <Full_Path_to_ArchiveAppConfig.xml> /PURGE /INCIDENTSBEFORE <Date_and_Time>
  ```

  For example:

  ```
  ArchiveApp.exe /CONFIG "%IDMS_ROOT%\config\server\ArchiveAppConfig" /PURGE /INCIDENTSBEFORE 2022-03-22T00:00:00
  ```

  This command deletes data from the INCIDENT table and all tables where the associated incident data is provided (the data is identified by INCIDENT_ID): INCIDENT_CHRONOLOGY, DAMAGE_ASSESSMENT, INCIDENT_CUSTOM, SWITCHING_STEP_INC_CUSTOM, SWITCHING_STEP_INCIDENT, RESTORATION_STEP, AFFECTED_PREMISES, CALL, INCIDENT_TIME_METRIC_EVENT, FOLLOW_UP_WORK_ORDER, ORDERS, ORDER_CUSTOM, DAMAGE, DEVICE, DEVICE_FEEDER, DEVICE_SERV_CENTER, DEVICE_STATION, and DEVICE_OPERATIONS.

  Also, near real-time incident data is deleted from the CREW_ACTIVITY and ITR_ETR tables, as appropriate.

> **Important**

> Purging does not affect normal working of the Archive application. If an instance of the Archive application is already running in "normal" mode, executing the purging command runs another instance of the Archive application, which shuts down after purging is completed.

4.3. Message Log

The below image shows the Archive application message log form at startup. It contains up to five tabs, one for each category of data to archive. There is an execution thread that corresponds to each category of data to archive and the Threads menu that shows the status (started or stopped) of each thread. Each log message is preceded by a timestamp. The startup log messages indicate that the thread has started and opened a database connection. Significant configuration parameter values are also logged.

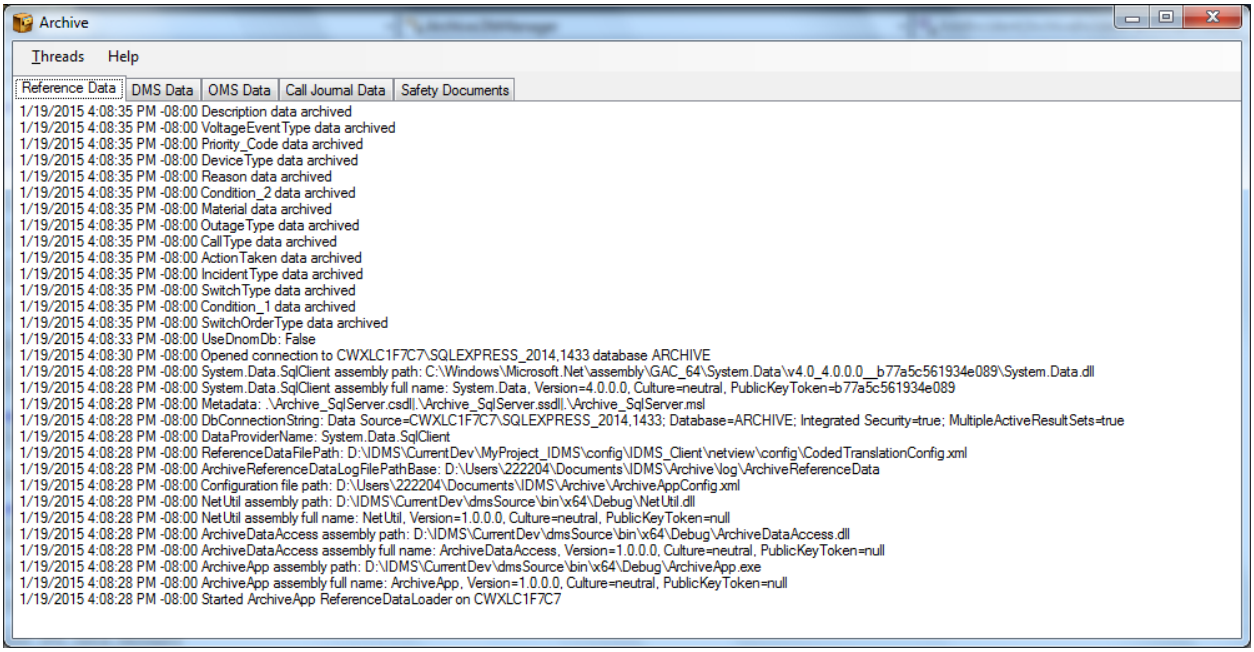


Figure 4. Archive Application Message Log

When data is archived, an appropriate message is logged. For example, when incident 201800016 is cleared, the message "Incident 201800016 archived" is logged on the OMS Data tab page. Messages are logged for the most significant data records (for example, incidents) but not for each component (for example, incident chronology log entries).

Each message written to the message log form is also written to a log file. There is one log file per tab page on the message log form. The log file paths are specified by the following configuration parameters:

- ArchiveReferenceDataLogFilePath
- ArchiveDmsDataLogFilePath
- ArchiveOmsDataLogFilePath

- ArchiveSafetyDocumentsLogFilePath

> **Note**
>
> The MESSAGE_LOG table can accumulate a large number of records over time, which may impact system performance. Regular maintenance, such as clearing, archiving, or purging old records, is required to manage table size.

4.4. Log Messages

This section provides detailed explanations of each Archive application log message. Words enclosed in angle brackets (for example, <path>) represent parameters that are replaced with values at run time.

4.4.1. <message>

This is a general error message template in which the entire message is determined at run time. When an exception occurs, the exception message is logged. This message represents a placeholder for any Microsoft .NET or Archive application exception message.

4.4.2. <reference data code set> data archived

This informational message indicates that a set of reference data was successfully archived. The ReferenceDataCodes configuration parameter lists the reference data codes to import from CodedTranslationConfig.xml. This message is logged for each element of that list.

For example, if ReferenceDataCodes is set to "Material, Reason, ActionTaken, DeviceType, IncidentType, OutageType, and SwitchOrderType" (itsdefault value), the above image shows the resulting log messages.

4.4.3. <configuration parameter name>: <value>

This informational message identifies the value of selected key configuration parameters. A few of these messages are logged when the Archive application starts, following the "Configuration file path" message. The above image shows this message displaying the values of the configuration parameters: ArchiveReferenceDataLogFilePath, ReferenceDataFilePath, DataProviderName, and Metadata.

4.4.4. Advanced application feeder parameters <id> archived

This informational message indicates that feeder parameters for an advanced application were successfully archived.

4.4.5. Advanced application power transformer parameters <id> archived

This informational message indicates that power transformer parameters for an advanced application were successfully archived.

4.4.6. Advanced application (<type>) plan <id> archived

This informational message indicates that an advanced application plan was successfully archived.

4.4.7. Advanced application (<type>) solution <id> archived

This informational message indicates that an advanced application solution was successfully archived.

4.4.8. Advanced application step <id> archived

This informational message indicates that an advanced application step was successfully archived.

4.4.9. Archive database schema version (<version>) is not compatible with this Archive application version (<version>)

This error message is logged upon startup if the Archive database schema version as specified in SYSTEM_PARAMETERS.CURRENT_VERSION is not compatible with the ArchiveApp assembly version. If the ArchiveApp assembly version number (for example, 3.2.0.0074) starts with the Archive database schema version number (for example, 3.2.0), it is deemed to be compatible. (The last part of the assembly version represents the build number.) This message typically indicates that a new version of ArchiveApp was installed, but the Archive database schema was not upgraded. In this case, the solution is to upgrade the database schema. For instructions on upgrading the database schema, refer to the *ADMS Relational Database Software Installation and Maintenance Guide*.

Note that this error message may not appear, and incompatibilities between the Archive database schema and application may not be detected at startup if changes to the Archive database schema or application do not result in version number updates. This is particularly likely when using prerelease installation kits, in which case the version numbers are for the anticipated next release. In this case, the incompatibility may result in runtime errors.

4.4.10. Archive journal file <file> moved to <path>

When an archive journal file is no longer being written to (for example, because a new file was

created for a new date), and all data from the archive journal file has been stored in the archive database, the Archive application moves the journal file to the directory specified by configuration parameter DmsDataArchivedPath (for DMS archive journals) or OmsDataArchivedPath (for OMS archive journals), and logs this informational message. Old archive journal files are moved to minimize the number of archive journal files that must be read in queries for new data to archive. This message does not pertain to call journal files, which are "retired" by Call Server, not by Archive.

4.4.11. ArchiveApp assembly path: <path>

This informational message identifies the path to the Archive application assembly, `ArchiveApp.exe`.

4.4.12. ArchiveDataAccess assembly path: <path>

This informational message identifies the path to the Archive data access assembly, `ArchiveDataAccess.dll`.

4.4.13. Archiving a batch of <count> journal entries

This informational message is displayed when the Archive application begins archiving many journal entries, indicating that it may take a few seconds to complete processing the batch. This typically occurs when the Archive application is started after a long downtime, when historical and near real-time data records to be archived have accumulated in the journals.

4.4.14. Call <id> archived

This informational message indicates that a call was successfully archived.

4.4.15. Closed connection to <data source> database <database>

This informational message is logged when an Archive application thread closes a database connection. This is normally logged only when the Archive application is shutting down. The data source and database names included in this message are derived from the DbConnectionString configuration parameter.

4.4.16. Configuration file (<name>) is invalid. <error>

This message is displayed if the Archive application is unable to deserialize (read) the `ArchiveAppConfig.xml` file. This typically indicates that the file contains malformed XML, such as a missing ">" in an element tag. The error message provides details of the problem.

4.4.17. Configuration file not found: <path>

If the configuration file path that is specified by the /CONFIG argument is not found by the Archive application, this error message appears. This occurs if the file does not exist or the file is not visible to the Archive application.

This error message appears in a message box as shown in the below image rather than in the Archive application message log because the Archive application cannot start without a valid configuration file.

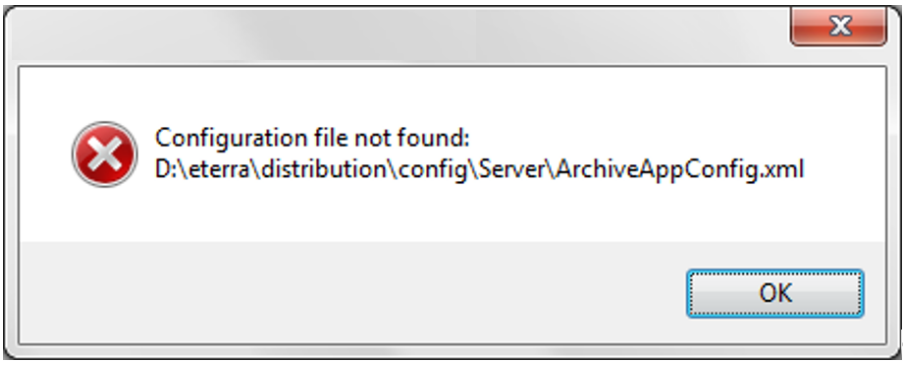


Figure 5. Configuration File Not Found Error

4.4.18. Configuration file path: <path>

This informational message identifies the path to the configuration file, ArchiveAppConfig.xml, as specified in the command-line argument used to start the Archive application.

4.4.19. Configuration is null. The configuration file (<path>) may contain invalid XML.

If the configuration file exists but cannot be deserialized, this error message appears. This occurs if the file contains invalid XML.

This error message appears in a message box as shown in the below image rather than in the Archive application message log because the Archive application cannot start without a valid configuration file.

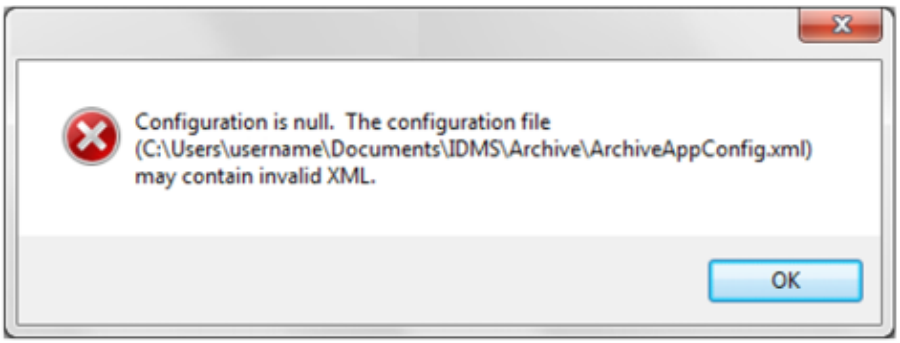


Figure 6. Configuration Is Null Error

4.4.20. Creating directory <path>

When a safety document is archived, the safety document archive file that is moved to the directory is specified by the configuration parameter SafetyDocumentsArchivedPath. If that directory does not exist, the Archive application attempts to create it and logs this informational message. Similarly, if the directory that is specified by the configuration parameter SafetyDocumentsNotArchivedPath does not exist, the Archive application attempts to create it and logs this message for that directory.

4.4.21. Data provider assembly path: <path>

This informational message identifies the path to the data provider assembly, `Oracle.ManagedDataAccess.dll` or `Oracle.DataAccess.dll` for Oracle, or `System.Data.dll` for SQL Server.

4.4.22. Directory not found: <path>

If the directories specified by the configuration parameters DmsDataPath, OmsDataPath, or SafetyDocumentsToArchivePath do not exist, this error message is logged. Unlike the directories that are specified by SafetyDocumentsArchivedPath and SafetyDocumentsNotArchivedPath, the directories that are specified by these configuration parameters contain inputs to the Archive application. The Archive application does not attempt to create these directories if they do not exist.

4.4.23. DPF feeder losses <id> archived

This informational message indicates that calculated real and reactive power losses by feeder for a Distribution Power Flow (DPF) solution were successfully archived.

4.4.24. DPF measurement island <id> archived

This informational message indicates that a measurement island for a Distribution Power Flow (DPF) solution was successfully archived.

4.4.25. DPF power transformer losses <id> archived

This informational message indicates that calculated real and reactive power losses by power transformer for a Distribution Power Flow (DPF) solution were successfully archived.

4.4.26. DPF solution <id> archived

This informational message indicates that a Distribution Power Flow (DPF) solution was successfully archived.

4.4.27. EntityValidationErrors: <error>

This error message should never be encountered. If data is unable to be stored in the Archive database, this multiline message will provide details about the cause of the problem. Report this error to GE.

4.4.28. Error creating directory <path>

If an error is encountered when the Archive application attempts to create a directory that is specified by the configuration parameters SafetyDocumentsArchivedPath and SafetyDocumentsNotArchivedPath, this error message is logged.

4.4.29. Error writing to file <path>

This error message is logged if an error is encountered when the Archive application attempts to write to a file.

4.4.30. Fault Location result <id> archived

This informational message indicates that a Fault Location result was successfully archived.

4.4.31. Fault Location solution <id> archived

This informational message indicates that a Fault Location solution was successfully archived.

4.4.32. Feeder Exception <id> archived

This informational message indicates that a feeder exception was successfully archived.

4.4.33. File not found: <path>

If the file specified by the configuration parameter ReferenceDataFilePath does not exist, this error message is logged.

4.4.34. Incident <name> archived

This informational message indicates that an incident is successfully archived.

> **Note**
>
> A single incident is represented by multiple records in the OMS archive journal. This log message indicates that an ArchiveIncident record has been archived, but the associated ArchiveIncidentChronology, ArchiveIncidentUpdate, ArchiveRestorationStep, ArchiveSwitchingStep, and ArchiveSwitchingStepRestorationStep records have not yet been archived. To limit the number of log messages, they are not written for all of these "child" records.
> Sometimes you may see many "incident archived" and "switching order archived" log messages followed by a few seconds in which no log messages are written. Incident and switching order child records (for example, ArchiveIncidentChronology) are archived during these gaps, even though no log messages are written when they are archived.

4.4.35. Invalid configuration parameter name: <name>

This error message should never be encountered. As explained in <configuration parameter name>: <value>, the values of selected key configuration parameters are logged when the Archive application starts. If the Archive application attempts to log the value of a configuration parameter that does not exist, this message is logged. Report this error to GE.

4.4.36. Meter event <id> archived

This informational message indicates that a meter event was successfully archived.

4.4.37. Meter status <id> archived

This informational message indicates that a meter status (response to a meter ping request) was successfully archived.

4.4.38. Meter voltage event <id> archived

This informational message indicates that a meter voltage event was successfully archived.

4.4.39. Meter voltage status <id> archived

This informational message indicates that a meter voltage status (response to a meter voltage ping request) was successfully archived.

4.4.40. Missing or invalid configuration parameter: <name>

If a required configuration parameter is not specified in the configuration file, this error message is logged.

4.4.41. Network model file info <id> archived

This informational message indicates that a Network Model File Info record was successfully archived.

4.4.42. New row inserted into table <name>: <row id>

If inserting a record into the Archive database would result in a foreign key constraint violation (referencing an object that does not exist in the database), the Archive application inserts a row into the parent (referenced) table in order to enable the row to be inserted into the child (referencing) table. For example, if a device to be archived references a new feeder that does not yet exist in the Archive database, the Archive application will insert the new feeder, and then insert the device. When rows are inserted in order to maintain referential integrity, this message is logged. It specifies the parent table into which a row was inserted, and an identifier of the row inserted.

4.4.43. Opened connection to <data source> database <database>

This informational message is logged when an Archive application thread successfully opens a connection to the database that is specified by the DbConnectionString configuration parameter. The above images show this message logged upon startup for two different Archive application threads. For security, the database user (schema) name is excluded from this message.

4.4.44. Reference data file not found: <path>

This error message indicates that the reference data file specified by the configuration parameter

ReferenceDataFilePath is not found.

4.4.45. Reference data was not archived

When the Archive application starts, if the configuration parameter ArchiveReferenceData is set to "true", reference data is archived first, before DMS data, OMS data, or safety documents. This ensures that the reference data tables are loaded before any historical data with foreign key references to those tables is stored. If a problem prevents the reference data from being archived, the threads that archive DMS data, OMS data, and safety documents are stopped, and this message is logged. If this happens, refer to the Reference Data tab page for information about the error and resolve that error before restarting the Archive application.

4.4.46. Safety document <name> archived

This informational message indicates that a safety document was successfully archived.

4.4.47. Safety document file <name> could not be moved to <path>

If a safety document cannot be archived because of errors, the Archive application attempts to move the file to the folder that is specified by the configuration parameter SafetyDocumentsNotArchivedPath. If the move operation fails, this error message is logged, and the Archive application shuts down.

4.4.48. Safety document file <name> moved to <path>

If a safety document cannot be archived because of errors, the Archive application attempts to move the file to the folder that is specified by the configuration parameter SafetyDocumentsNotArchivedPath. If the move operation succeeds, this informational message is logged, and the Archive application continues processing.

4.4.49. Safety document file <name> not archived

This error message indicates that a safety document could not be archived because of errors. It is preceded by a message that describes the cause of the archiving failure. In this case, the Archive application attempts to move the safety document archive file to the folder specified by the configuration parameter SafetyDocumentsNotArchivedPath. If the move operation succeeds, the Archive application continues processing. If the move operation fails, the Archive application shuts down, so that it does not continuously attempt to archive the safety document.

4.4.50. Shutting down due to errors

When the Archive application encounters errors from which it cannot recover, it logs the error messages followed by the message (that is, Shutting down due to errors), and then shuts down.

4.4.51. Started ArchiveApp <thread name> on <computer name>

This informational message indicates that an Archive application thread is started.

4.4.52. Station Exception <id> archived

This informational message indicates that a station exception was successfully archived.

4.4.53. Stopped ArchiveApp <thread name> on <computer name>

This informational message indicates that an Archive application thread is stopped.

4.4.54. Switching order <name> (state = <state>) archived

This informational message indicates that a switching order was successfully archived. A switching order is archived before its steps.

4.4.55. System.Transactions.TransactionManagerCommunicationException

Error: System.Transactions.TransactionManagerCommunicationException: Network access for Distributed Transaction Manager (MSDTC) has been disabled. Please enable DTC for network access in the security configuration for MSDTC using the Component Services Administrative tool.

Solution: Use the following articles to enable network access and configure Microsoft Distributed Transaction Coordinator (DTC), as appropriate.

- https://docs.microsoft.com/en-us/biztalk/core/troubleshooting-problems-with-msdtc?redirectedfrom=MSDN#if-windows-firewall-is-running-add-an-exception-for-the-msdtc-service
- https://docs.microsoft.com/en-us/troubleshoot/windows-server/application-management/configure-dtc-to-work-through-firewalls

- https://stackoverflow.com/questions/5149591/network-access-for-distributed-transaction-manager-msdtc-has-been-disabled/28636625#28636625

4.4.56. Unable to close connection to <data source> database <database>

This error message is logged if the Archive application is unable to close a connection to the database. If a database connection is successfully opened, it generally can be closed. If a database connection cannot be closed, it is probably due to a temporary network problem.

4.4.57. Unable to open connection to <data source> database <database>

This error message is logged if the Archive application is unable to open a connection to the database using the connection string that is specified by the DbConnectionString configuration parameter. Verify that a database connection can be opened from the application server without running the Archive application, such as via SQL*Plus. If you cannot connect to the database, resolve the database or network connectivity failure. If you can connect, the DbConnectionString setting is probably incorrect. It is also possible that the data provider assembly is incorrect. It may be a 32-bit assembly rather than the required 64-bit assembly.

4.4.58. Unexpected caller of method <called method>: <calling method>

This error message should never be encountered. If the Archive application determines that a method was called from another method that it did not expect, this message is logged. Report this error to GE.

4.4.59. Unknown attribute: <name>

This error message indicates that the configuration file (`ArchiveAppConfig.xml`) contains an XML element with an attribute that is not an expected attribute of the Archive application configuration. This is a configuration error that must be corrected before the Archive application will run.

4.4.60. Unknown element: <name>

This error message indicates that the configuration file (`ArchiveAppConfig.xml`) contains an XML element that is not an expected element of the Archive application configuration. This is a configuration error that must be corrected before the Archive application will run.

This may occur after installing a new version of the Archive application if a configuration parameter that was valid in the old version is no longer valid in the new version.

4.4.61. Unrecognized reference data: <name>

This error message should never be encountered. The list of reference data codes to be archived is hard-coded in the Archive application. If that list includes a reference data code that the Archive application is unable to archive, this message is logged. Report this error to GE.

5. Sample Reports

ADMS provides some sample distribution and outage management reports that use near real-time and historical data from the Archive database. Power companies may use these reports directly, or as templates for their own custom reports.

- The `ADMS_sample_reports.msi` installer includes sample distribution and outage management reports using historical data from the Archive database. These reports are implemented using Microsoft SQL Server Reporting Services (SSRS), which is a feature of MS SQL Server. The reports are accessible from any web browser.

  For more information about the ADMS Reports application, refer to the following:

  - Reports Software Installation and Maintenance Guide
  - Reports User Guide

- The `ADMS_relational_database.msi` installer includes sample reports for near real-time data. These reports are implemented using the Microsoft Power BI application, which allows generating various types of reports from different data sources, such as Excel or CSV files, or MS SQL Server database. The reports can be viewed by different types of users or user groups on the network or on the internet.

  For more information about Power BI reports, refer to the *Relational Database Software Installation and Maintenance Guide*.

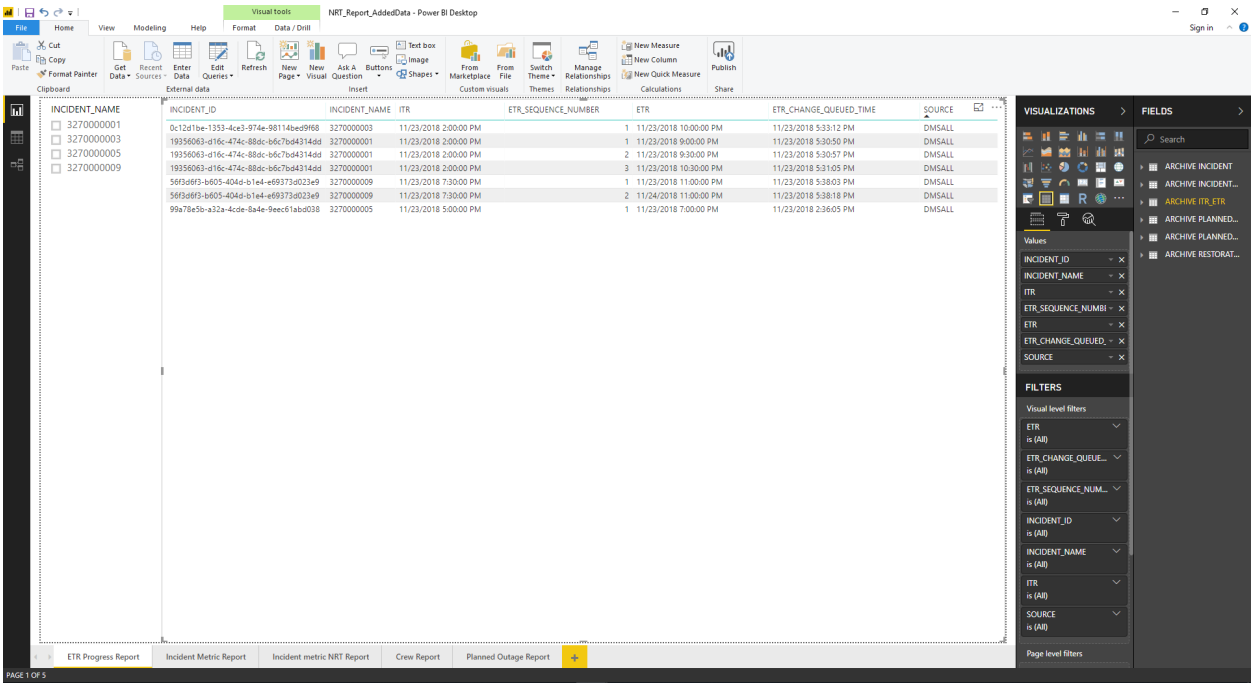


Figure 7. ETR Progress Report in Microsoft Power BI

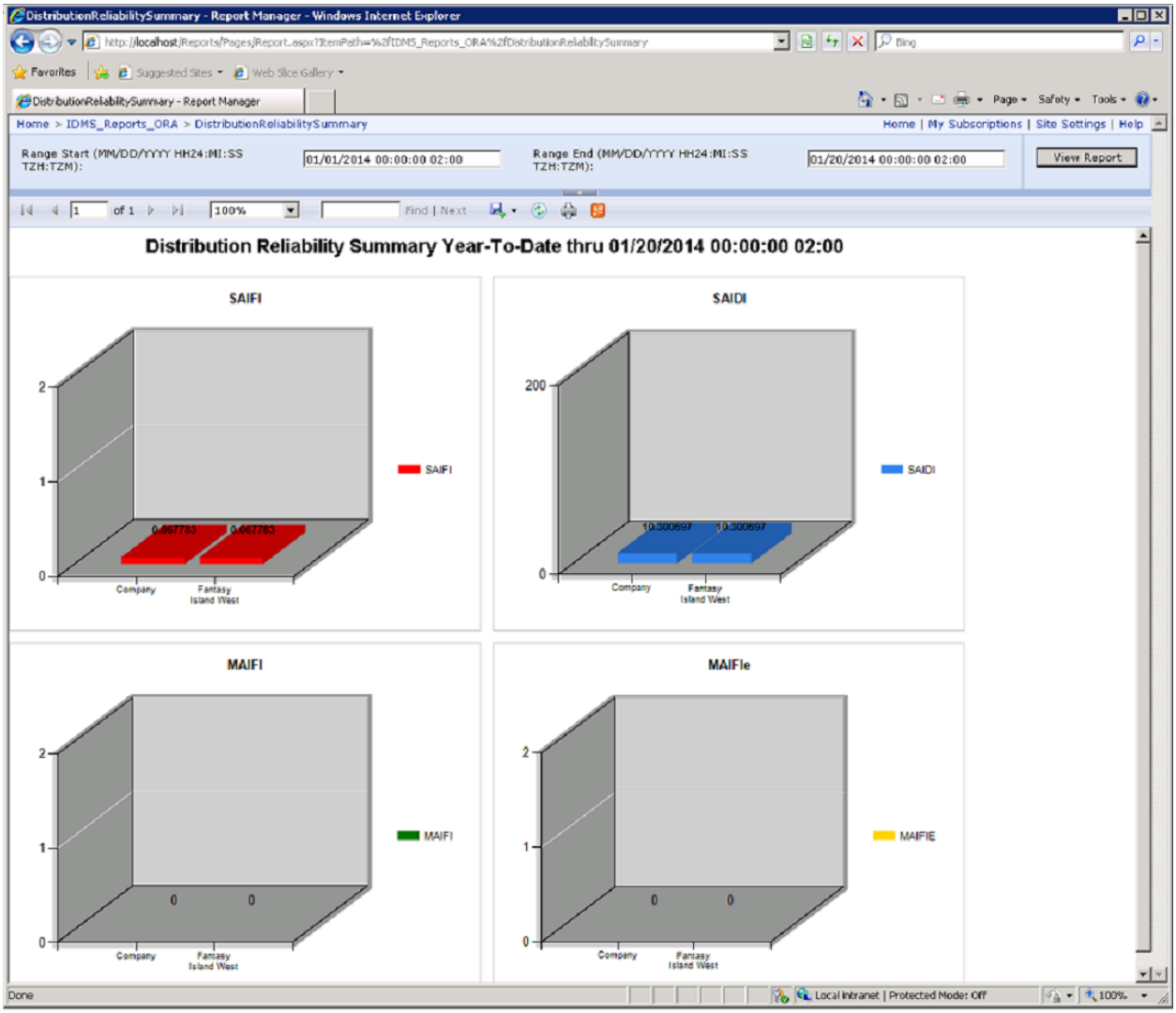


Figure 8. Distribution Reliability Summary Report in SSRS

6. DNOM Database Loader

The ADMS DNOM Database Loader is a program that opens and reads binary station model files and loads the data into the DNOM database, which is an Oracle or a MS-SQL relational database. The DNOM Database Loader can run as a Windows service or as a command-line program. It monitors a directory for new station model files specified in the DNOM Database Loader Configuration Editor. Once station model files are placed in the monitored directory, the DNOM Database Loader will upload the model file data into the DNOM database.

6.1. DNOM Database Loader Prerequisites for Oracle and SQL Server

Before using the DNOM Database Loader with an Oracle database, you must install the Oracle client software package described in Installing Oracle Data Provider for .NET (ODP.NET). Both for Oracle and MS SQL Server, you must install the corresponding DNOM Database Schema and Packages. For detailed information about the DNOM database schema and package installation, refer to the *ADMS Relational Database Software Installation and Maintenance Guide*.

6.2. Installing the DNOM Database Loader

The DNOM Database Loader is installed with the ADMS Tools installer.

The following shortcuts are created in the Eterra> Distribution> Tools> Modeling Tools folder:

- DNOM DB Loader Application: Starts the loader command-line program
- DNOM DB Loader Service Manager: Starts the loader service manager, and configures and manages loader services

One additional shortcut is created in the Eterra> Distribution> Tools> Configuration Tools folder:

- DNOM DB Loader Configuration Editor: Starts the database loader configuration tool.

6.3. Configuring DNOM Database Loader Parameters

To configure the DNOM Database Loader parameters, from the Windows Start menu, navigate to Eterra> Distribution> Tools> Configuration Tools> DNOM DB Loader Configuration Editor.

The configuration parameters are divided into two subgroups: "Database Connection Parameters" and "Configuration Parameters".

To configure the Database Connection Parameters:

1. Choose a provider from the drop-down list, that is either "Oracle" or "MSSQL".

2. Enter the database user name and password in the User Name and Password text boxes.
3. Enter the Database Address Name. The name defines the database listener and service name used to access the database.

   For SQL Server, the format is //host/database_name.

   For Oracle, the format is //host:port/service_name.
4. You can test your connection to the Database by clicking the "Test Connection" button. If the Configuration Editor can successfully establish a connection using the parameters you entered, a dialog box will appear with the message "Database connect successfully."
5. When you have successfully configured the database connection parameters, save the configuration and then continue with defining the configuration parameters.

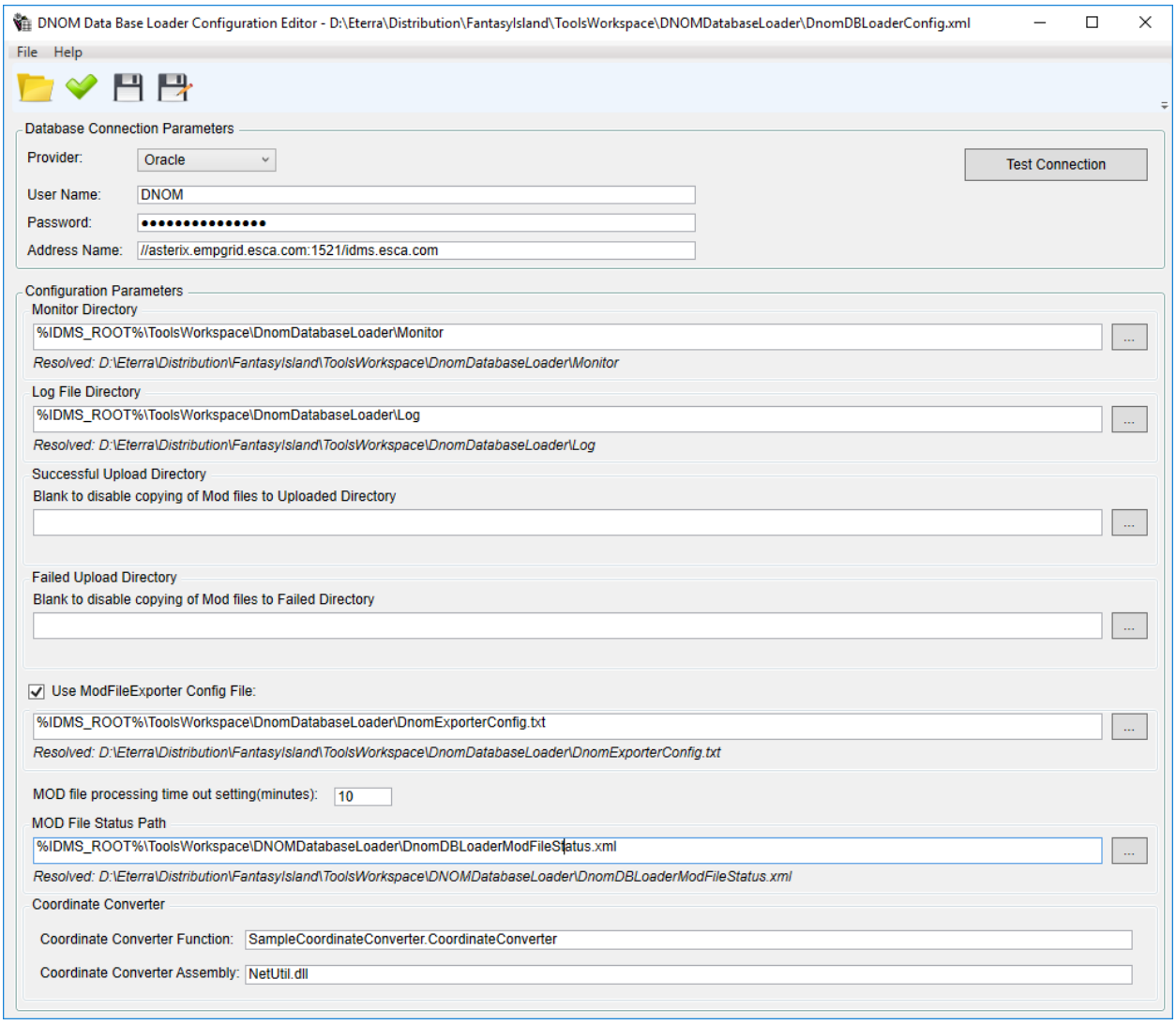


Figure 9. DNOM DB Loader Configuration Editor

You can configure the parameters that control where the DNOM DB Loader looks for model files to upload to the database the Configuration Parameters group.

The following fields can be configured in the Configuration Parameters group:

- **Monitor Directory:** The directory where station model files should be placed for loading into the DNOM database.

  > **Caution**
  >
  > The Monitor directory must always contain an actual set of model files. When a model file is removed from the Monitor directory, its model data is removed from the DNOM database.

- **Log File Directory:** The location where log files are stored.
- **Successful Upload Directory:** Specifies the directory where the MOD files are moved after successfully loading into the DNOM database. To disable MOD file copying, leave the field blank (recommended).
- **Failed Upload Directory:** Specifies the directory where MOD files are moved after an unsuccessful attempt at loading the files into the relational database. To disable MOD file copying, leave the field blank (recommended).

  This option is useful when you need to view which MOD file failed to upload. You can also view the latest status for each MOD file in the DnomDBLoaderModFileStatus.xml file stored in the %IDMS_ROOT%\ToolsWorkspace\DNOMDatabaseLoader directory.

- **Use ModFileExporter Config File:** Select this check box if you want to define the record types that are extracted from the MOD file and loaded into the relational database (recommended). For information on how to define the set of included record types, see the example file (%IDMS_ROOT%\ToolsWorkspace\DNOMDatabaseLoader\DnomExporterConfig.txt) included with the kit.
- **MOD File Processing Time Out Setting (Minutes):** The time after which the loading process aborts the current model file load attempt so that it does not block loading the next model file.
- **MOD File Status Path:** Specifies the path of the file in which the model export status will be written.
- **Coordinate Converter:** Parameters for converting system coordinates (X, Y) into Projected Coordinate System (PCS) coordinates (latitude, longitude).
  - **CoordinatesConverterAssemblyName:** Name of the assembly (DLL) containing a class that converts system X/Y coordinates into latitude/longitude coordinates.
  - **CoordinatesConverterClassName:** Name of the class that converts system X/Y coordinates into latitude/longitude coordinates.

  > **Note**
  >
  > For US customers, the default coordinate converter can be used. For non-US customers, a custom converter must be developed.

If you make any changes, click Save before exiting the Configuration Editor.

6.4. DNOM DB Loader Service Manager

You may need to operate more than one DNOM DB Loader service (for example, to load model data into different regional databases). The DNOM DB Loader Service Manager allows you to configure multiple DNOM DB Loader services and provides an easy method that you can use to start and stop these services. It also provides the same configuration interface as the Configuration Editor so you can use it to configure the various loader configuration parameters for the services you create.

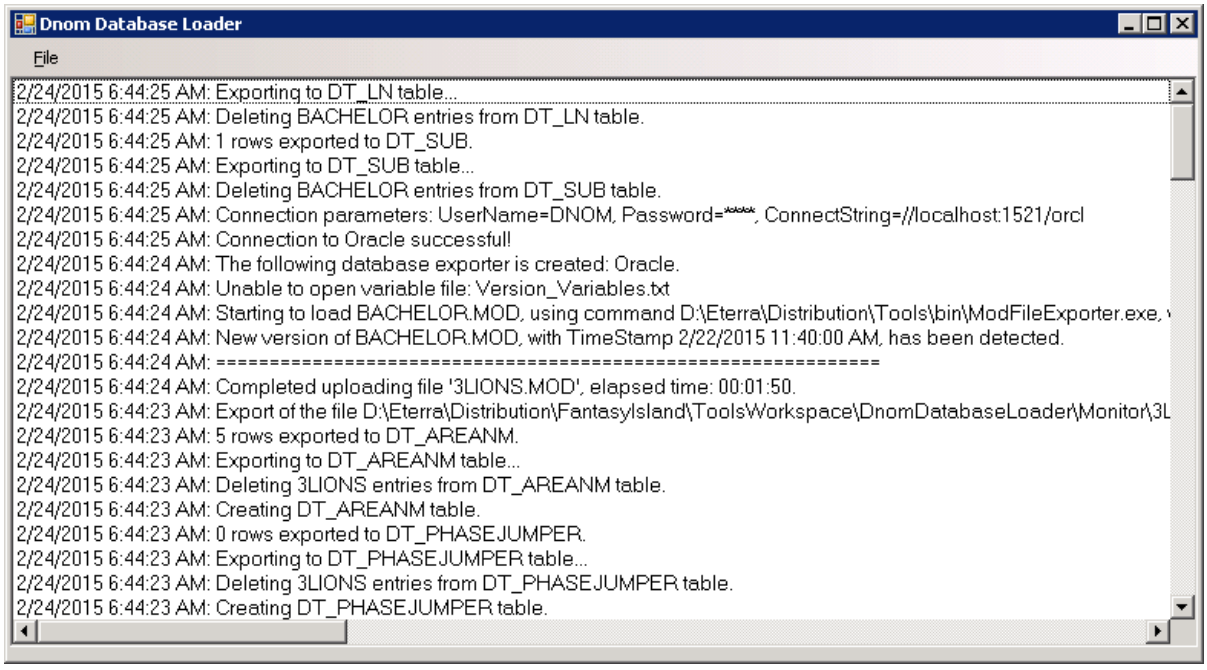
> **Note**
>
> By Default, the logon account for the DNOM DB Loader service is set to NetworkService.

6.5. Starting and Stopping the DNOM Database Loader

You can use one of the following ways to start the DNOM DB Loader:

- To start the loader as a command-line program from the Windows Start menu, navigate to All Programs> Eterra> Distribution> Tools> Modeling Tools> DNOM DB Loader Application.

  When you start the loader using this method, the DNOM Database Loader application opens the following window:

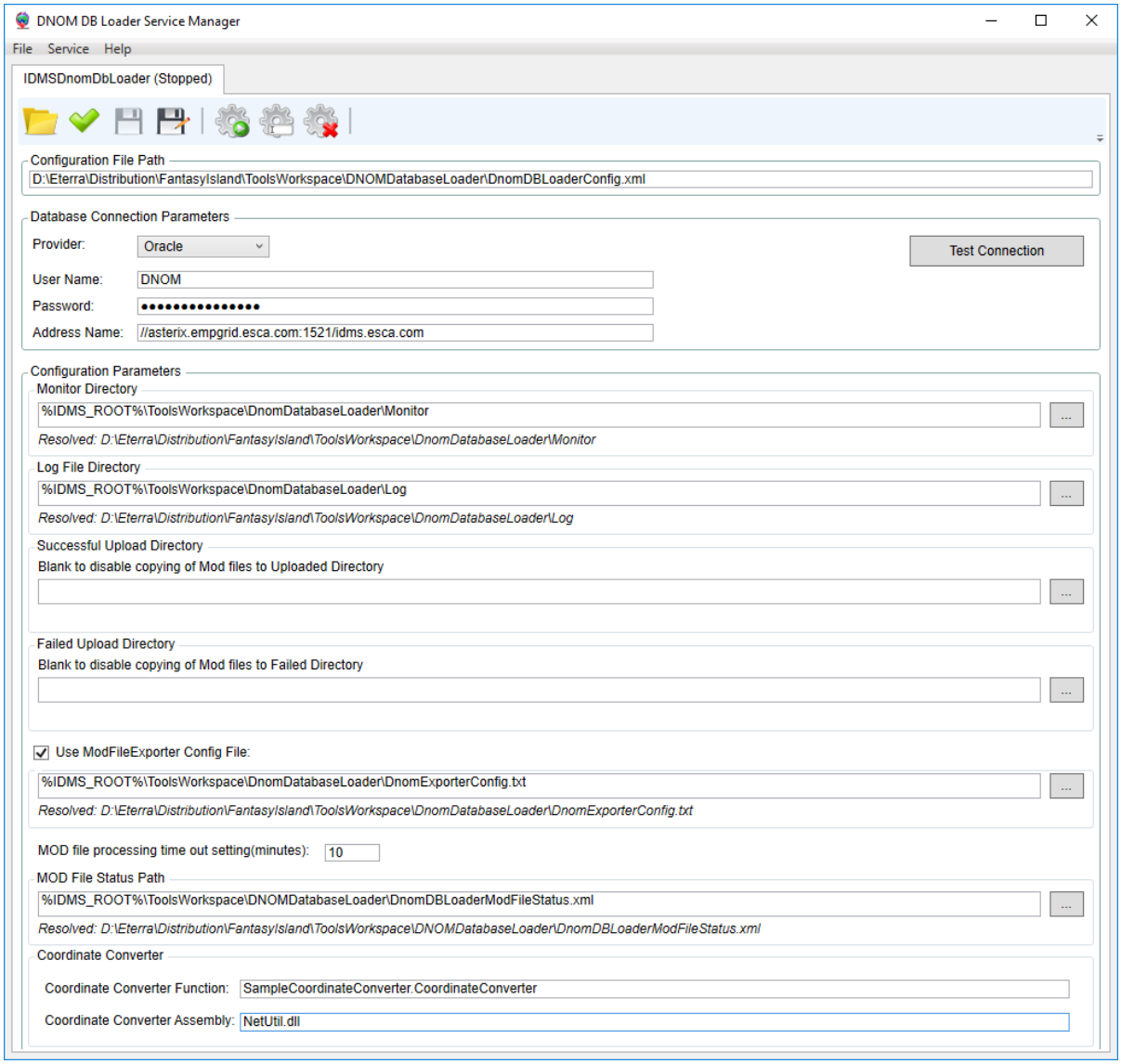


  The loader checks the configured "Monitor" folder, waiting for new Mod files. As new Mod files are processed, progress is logged in this window. Closing this window stops the loader.

- To start the loader as a Windows service in the Service Manager, from the Windows Start menu:
  - Navigate to All Programs> Eterra> Distribution> Tools> Configuration Tools> DNOM DB

Loader Service Manager.

- In the Service Manager, select the tab at the top that corresponds to the service you want to start, and then click the Start button on the toolbar.



To stop the service:

- From the Service Manager, select the tab at the top that corresponds to the service you want to stop and click the Stop button on the toolbar.

Regardless of how the loader is started, it logs its progress in a log file in the configured Log folder. The log file names are date stamped. A new log file is created at 12:01 a.m. each day. When running the loader as a service, the only way to monitor its progress is to view the contents of the most recent log file.

6.6. Disabling Windows Problem Reporting

The DNOM Database Loader works by starting another process (`ModFileExporter.exe`) to read the model file and write to the database. This process is managed by the DNOM Database Loader as a subprocess. If the ModFileExporter process has problems, it could hang the Loader and stop the processing of model files.

To limit the chances of the ModFileExporter negatively affecting the Loader, you may consider disabling Windows problem reporting on the host where the DNOM Database Loader is running. Disabling Windows problem reporting prevents the problem reporting feature from interfering with the executing service if the service encounters an error while running the subprocess that parses the MOD file and writes to the DNOM Database.

To disable Windows problem reporting:

1. Open the Control Panel and click Action Center.
2. On the left panel, click Change Action Center Settings.
3. On the bottom, click Problem Reporting Settings.
4. Select the "Never Check for Solutions" option, and then click the OK button.

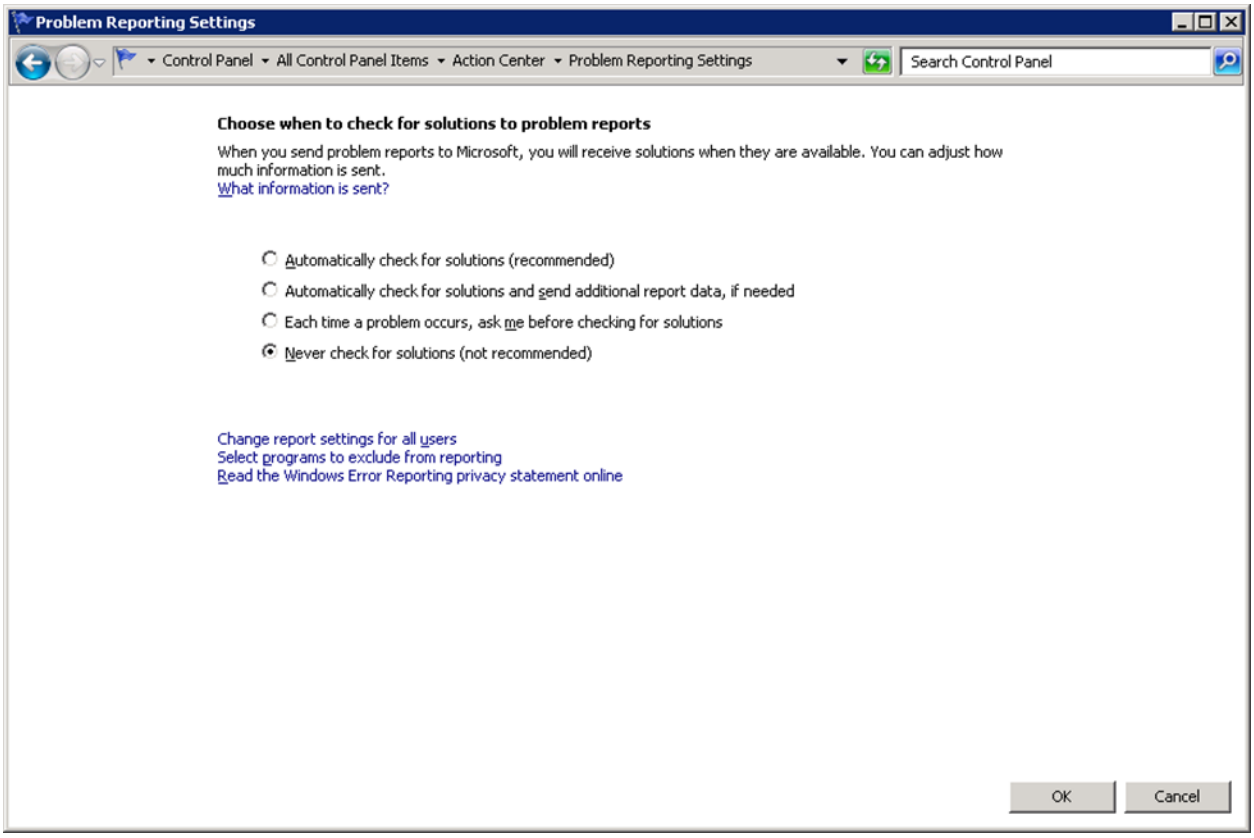


Figure 10. Windows Problem Reporting Settings

6.7. DNOM Database Loader Logging

The DNOM Database Loader logs informational and error messages to a log file in the %IDMS_ROOT%\ToolsWorkspace\DNOMDatabaseLoader\Log directory. The file is named "DnomDBLoaderLog" and is date stamped (for example, DnomDBLoaderLog_2014-07-18.log). A new file is created each day when the first write IO is attempted after 12 a.m.

Errors encountered by the DNOM Database Loader and its subprocess, ModFileExporter, are logged in the log file. Errors are written with the word "Exception" in the output line. If you want to monitor the health of the DNOM loading, you could design a process to scan the daily log file for errors.

The operator can use the Windows Event Viewer to check the DNOM Database Loader's service event log when necessary. If the DNOM Database Loader has trouble starting, it logs informative messages to the Windows Event Log.

The Event Viewer can be accessed via the Control Panel> Administrative Tools> Event Viewer. The DNOM Database Loader's log can be found in the Applications and Services Logs folder and is labeled "eterradistribution DnomDBLoader Log".

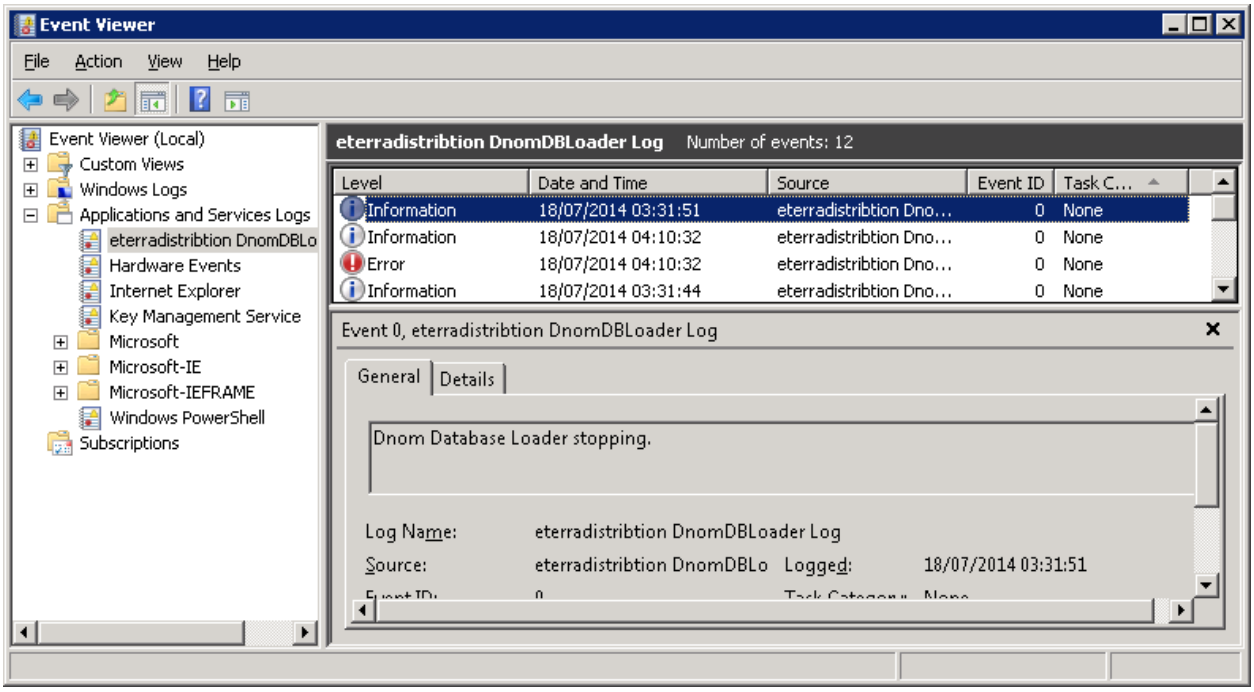


Figure 11. Event Viewer

6.8. Defining Tables to Load

The DNOM Database Loader uses a ModFileExporter.exe process to open, read, and load model files. The ModFileExporter uses command-line arguments to point to the database host and service and to the optional configuration file (DnomExporterConfig.txt) that defines a list of database tables

and columns to load into the DNOM database. The default configuration file is installed with the Tools kit at %IDMS_ROOT%\ToolsWorkspace\DNOMDatabaseLoader\DnomExporterConfig.txt. By default, the ModFileExporter is configured to load a subset of all the tables defined in the model file. This subset is the set of tables that are needed by ADMS Archive.

> **Tip**
>
> You can modify DnomExporterConfig.txt to specify which tables will be loaded.

In the DNOM Database Loader Configuration dialog box, the "Use ModFileExporter Config File" option defines whether the table configuration file `DnomExporterConfig.txt` is used. If not selected, the ModFileExporter uses settings similar to those specified in the default `DnomExporterConfig.txt` file.

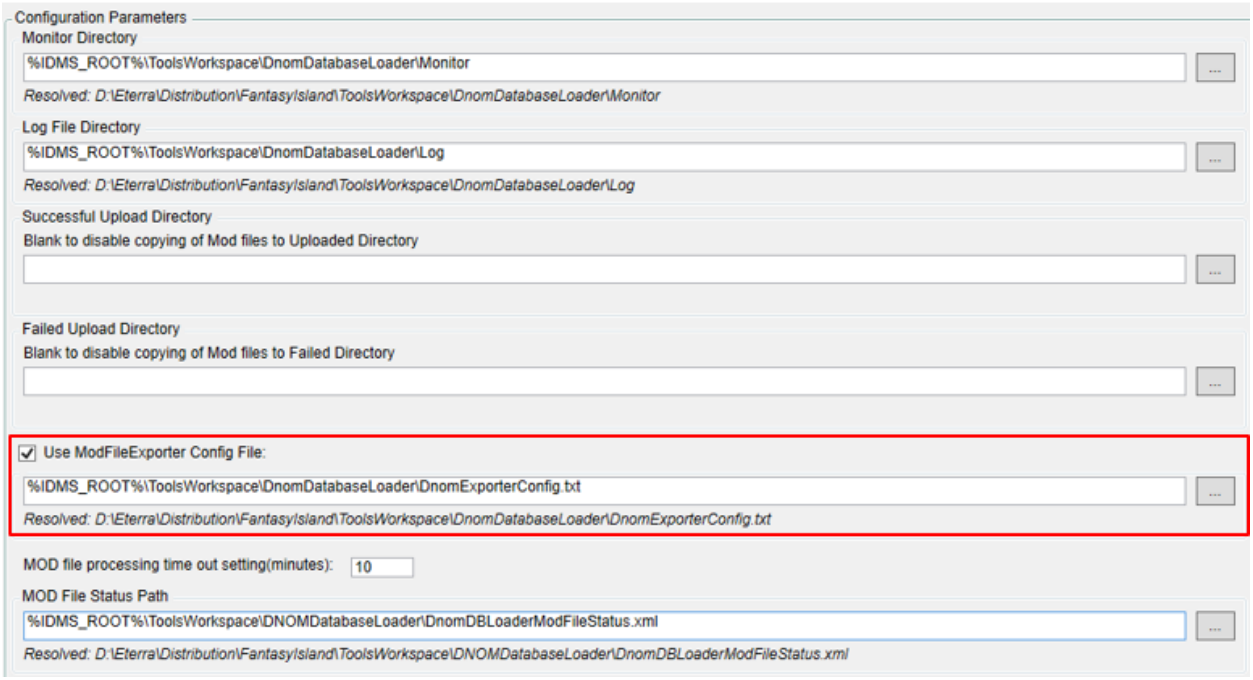


Figure 12. Using DnomExporterConfig.txt to Define Tables to Load

The `DnomExporterConfig.txt` file contains the following parameters:

- **LOG_TO_FILE:** Specifies whether the ModFileExporter_<Timestamp>.log file is created in %IDMS_ROOT%/log.
- **EXPORT_TABLE_PREFIX:** The table name prefix that the ModFileExporter uses when creating tables in the database. For example, the "DT_" prefix defines that the SUB table name is set to DT_SUB. If this parameter is missing in the configuration file or is blank, the default value, "DT_", is used.
- **EXPORT_TABLE_COLUMN_PREFIX:** The column name prefix that the ModFileExporter uses when creating table columns in the database. For example, the "C_" prefix defines that the ID column name is set to C_ID. If this parameter is missing in the configuration file or is blank, the default value, "C_", is used.

- **EXPORT_COLUMN_SEQID_USE:** Defines whether the DNOM Database Loader adds the Sequential ID column to each DNOM table. Supported values are "yes" and "no". For more details, see Table Entry Sequential ID.
- **EXPORT_COLUMN_SEQID_NAME:** Defines the name for the Sequential ID column. If this parameter is missing in the configuration file or is blank, the default value, "SEQ_ID", is used.
- **EXPORT_TABLES:** The list of tables to be created and populated in the database. These names are analogous to the DNOM object names (except for the prefixes). The table prefixes must match the value defined by the EXPORT_TABLE_PREFIX parameter.

  For example:

  ```
  EXPORT_TABLES = DT_SUB, DT_LN, DT_MEAS
  ```

- **EXPORT_TABLE_COLUMNS.<TABLE_NAME>:** Defines whether a table is populated in the database. A separate EXPORT_TABLE_COLUMNS.<TABLE_NAME> parameter must be defined for each exported table that needs to be populated. The table names are analogous to the DNOM query field names (except for the prefixes). The table prefixes must match the value defined by the EXPORT_TABLE_PREFIX parameter.

  For example:

  ```
  EXPORT_TABLE_COLUMNS.DT_SUB
  EXPORT_TABLE_COLUMNS.DT_LN
  EXPORT_TABLE_COLUMNS.DT_MEAS
  ```

> **Note**
>
> In tables with C_DNOMX and C_DNOMY columns, ModFileExporter automatically creates C_LATITUDE and C_LONGITUDE columns and populates these columns with latitude and longitude coordinates converted from the system X and Y coordinates.

6.8.1. Table Entry Sequential ID

The Sequential ID feature is designed to show the DNOM table entry sequential ID. This feature is disabled by default. If enabled (EXPORT_COLUMN_SEQID_USE = yes in DnomExporterConfig.txt), the Sequential ID column is added to all DNOM tables and is populated automatically with the sequential number of an entry in the table. For example, the Sequential ID value is set to 1 for the first table entry and is set to 2 for the second table entry. The Sequential ID values are generated each time the DNOM Database Loader loads a station model (when loading a station model, the tool removes and re-inserts all table entries associated with the station).

To use the Sequential ID feature for a DNOM database that is already loaded with DNOM data:

1. Drop all database tables.
2. Configure the Sequential ID feature in DnomExporterConfig.txt.

For example, use the following configuration to enable the Sequential ID feature and name the sequential ID column "GreenEnergyID".

```
EXPORT_COLUMN_SEQID_USE = yes
EXPORT_COLUMN_SEQID_NAME = GreenEnergyID
```

3. Reload the DNOM database.

> **Note**
>
> The Sequential ID feature is based on the IDENTITY column feature, which is available in Oracle 12c and later and in MS SQL Server. In the MS SQL Server, only the IDENTITY column is supported as primary key. No other primary keys can be added to the tables.

7. Troubleshooting

7.1. Error Messages

When the Archive application encounters an error, an error message is logged in the message log. For example, an error message is logged if the Archive application is unable to open a connection to the Archive database because of an invalid DbConnectionString configuration parameter value. If the Archive application is unable to continue, it will shut down. In that case, the error messages will appear only in the log file, because the message log form that is displayed while the Archive application is running will be closed.

The ARCHIVE_WRITE_EXCEPTION table is used for backward compatibility. Older versions of the Archive application used this table to record error messages that occurred in legacy database stored procedures. The current version of Archive does not record new error messages in this table. However, the table may still contain old error messages.

Database errors that are encountered during an Oracle database schema upgrade, as opposed to during execution of the Archive application, are written to table UPGRADE_SPERRORLOG.

> **Note**
>
> "SP" in this table name refers to SQL*Plus, an Oracle database tool that is used to execute Oracle database schema upgrade scripts.

7.2. Archive Application Threads

If the Archive application cannot continue archiving data in any of its threads, the entire application shuts down, rather than stopping only the thread that encountered the error. The Process Manager may attempt to restart the Archive application, but the failure will be visible to system administrators monitoring the Process Manager.

If the problem affects only one thread, you can temporarily change `ArchiveAppConfig.xml` to not archive that category of data (for example, safety documents), while continuing to archive other data (for example, OMS data).

If a thread is enabled, you can manually stop a running thread by deselecting the check box next to the thread name on the Threads menu. Similarly, you can manually restart a stopped thread by selecting the appropriate checkbox on the Threads menu.

7.3. Invalid or Missing Data

Because the Archive database is designed to store actual system data, the data is not validated before it is archived. For example, if a safety document references an invalid device ID, the safety document must be archived anyway. The only data problem that prevents a record from being

archived is a missing primary key (for example, incident ID), but this is an application error, not a data error, and should not occur.

If data in the Archive database is invalid or missing, check the source record (for example, journal entry or safety document). If data is also invalid or missing in the source record, the record was archived correctly. Software changes may be necessary in the source application (for example, OMS) to prevent such invalid data or missing data errors in the future, but the Archive application or database may not need to be changed in that case.

The Archive database includes referential integrity constraints to maintain referential integrity (that is, to prevent references to objects that do not exist in the database). The Archive application prevents referential integrity violations by dynamically inserting records if necessary. For example, if an incident references a station that does not exist in the STATION table, a row is inserted into the STATION table with STATION_ID (primary key) set to a new GUID (Global Unique IDentifier) and STATION_NAME set to the station name specified on the incident.

7.4. Archiving and Journaling Exceptions

This section lists archiving exceptions and specific cases.

7.4.1. Journaling a Customer with Blank Account Number

Case: An incident that includes a customer with a blank account number (for example, an inactive customer) is written to an archive journal.

Exception: The account number value in the archive journal is written as "00000".

7.4.2. Journaling Momentary Incident Count

Case: A momentary incident is created and cleared in the system. The momentary count is recorded in the archive journal.

Exception: The momentary count is set anytime the incident duration (time between the de-energization and energization events) is less than the configured momentary incident duration. For an incident that includes multiple momentary outages that have the same device of record, the momentary counts are merged before the incident is cleared by the system.

7.4.3. Journaling Customer Device Information

For performance reasons, device information (for example, STATION, FEEDERID, DEVICEID) is not recorded in the Call journal and CALL table in the Archive database for customer calls that result in outage incidents (except for isolated incidents). The device information is only recorded for customer calls that result in isolated outage incidents, non-outage incidents, or unassociated calls.

7.5. Exceptions under which Archive Application Exits

This section lists all the possible errors because of which the Archive application exits.

7.5.1. Configuration File Doesn’t Exist

If the configuration file does not exist in the "%IDMS_ROOT%\config\server\ArchiveAppConfig.xml" path, the Archive application exits with the error message, "Configuration File not found".

7.5.2. Configuration File has Invalid Xml Element

If the Archive application configuration file has an invalid Xml element or Xml attribute, the Archive application exits with the error message, "Configuration is null. The configuration file may contain invalid XML.".

7.5.3. Error Creating Directory

If the "Archived" directory does not exist, the Archive application creates a directory (in the %IDMS_ROOT%\dynamics\server...\ path). If the logged in user doesn’t have privileges to create a directory, the Archive application exits with the error message, "Error creating directory".

7.5.4. Error Writing to File

The Archive application updates the log files while writing to database. If the logged in user doesn’t have write privileges, the Archive application exits with the error message, "Error writing to file".

7.5.5. Invalid Data Provider Name in the Config File

If the Data Provider name mentioned in the configuration file is not compatible with the Oracle or SQL Server databases, the Archive application exists with the error message, "Missing or invalid configuration parameter".

7.5.6. Invalid Values in Connection String or Database Doesn’t Exist

If the Connection String has invalid values or if the database name mentioned in the Connection String does not exist, the Archive application exists with the error message, "Unable to open connection to database".

7.5.7. Process Cannot Access the File

If another process is using the file that the Archive application is trying to write to, the Archive application exits with the error message, "The process cannot access the file because it is being used by another process".

7.5.8. Foreign Key Constraint Violation

Foreign key constraint violation may occur if the reference data look up is failed.

For example, if the "System Affected" table has only two records with the values for ID column as 0, 1 and if the Incident table is referring to ID 3 in the "System Affected" table, then the foreign key constraint violation might occur.

7.5.9. Config File Path not Mentioned in Purge Command

If the Configuration file path is not mentioned in the Purge command, the Archive application exits with the error message, "Argument Null exception".

7.5.10. Schema Mismatch

If there is a schema mismatch between the Archive application and database, the Archive application exits with the error message, "Invalid object name".

For example, the newly added "Orders" table, which is not in the database, but its schema is there in the Archive application data dictionary, then the Archive application exits with this error message.

> **Note**
>
> All the above error messages are logged to the log file located at the "%IDMS_ROOT%\log" path.

8. Major Event Day Calculations

This appendix provides details for the Major Event Day (MED) functionality, which is supplied with the Archive database.

Major Event Day calculations are supported by a set of specific database tables and stored procedures. This appendix describes MED calculations, implementation details, and required Archive database post-installation steps to enable the MED processing.

8.1. Functional Description

8.1.1. Determining MED, SAIDI, and MED_THRESHOLD

A given day is determined to be a major event day based on the aggregate customer minutes of outage interrupt of that day. The exact criteria used is defined in section 2.5 Beta Method in the *IEEE Guide for Electric Power Distribution Reliability Indices* (*IEEE Std 1366™-2012*) document (the standard).

Per the standard, a day is determined to be MED when its System Average Interruption Duration Index (SAIDI) exceeds a MED Threshold (called TMED in the standard, and MED_THRESHOLD in the MED_THRESHOLD table) defined for the interval (typically, a year) including that day, where:

- A day is a major event day when SAIDI > MED_THRESHOLD.

$$Daily\ SAIDI = \frac{\sum Customer\ minutes\ of\ interruption}{Total\ number\ of\ customers\ served}$$

- Only SAIDI for days where there was an interruption (that is, SAIDI is not equal to 0) are considered. Additionally, days during which catastrophic events occurred are excluded. (For more details about determining catastrophic event days, see Functional Aspects of MED Calculations.)
- MED_THRESHOLD is calculated based on the historical SAIDI over a period of 5 prior years.
- MED_ THRESHOLD = $e^{(\alpha+2.5\beta)}$, where:
  - α is an average of the natural logarithms of the SAIDIs.

$$\alpha = \frac{\sum \ln(SAIDI)}{Number\ of\ SAIDIs}$$

  - β is the standard deviation of the natural logarithms of the SAIDIs.

$$\beta = STDEV\,(\ln(SAIDI))$$

8.1.2. Functional Aspects of MED Calculations

Specifics of MED calculations are as follows:

- The standard presents SAIDI, MED_THRESHOLD, and MED as "System" level metrics. The GE implementation allows those metrics to be determined for subsets of the complete model. Those subsets are referred to as Beta Major Event Entities (BMEE).
- The mapping between incidents (outages) and BMEE is project-specific. The mapping is typically done via service centers: an outage is mapped to a service center, which in its turn is mapped to a BMEE.
- Catastrophic events days are currently determined on a BMEE basis as days for which SAIDI is greater than a given threshold. The SAIDI values for these days are excluded from the calculation (on a BMEE basis).
- The current implementation excludes incidents/restoration steps from the SAIDI calculation based on:
  - A list of incident cause codes defined in the MED_EXCLD_INCIDENT_CAUSE table.
  - A list of outage type codes defined in the MED_EXCLD_OUTAGE_TYPE table.
  - A list of device type codes defined in the MED_EXCLD_DEVICE_TYPE table.
  - Value of the IS_MOMENTARY field at the RESTORATION_STEP level.
  - Value of the EXCLUDE_FROM_STATISTICS field at the INCIDENT and RESTORATION_STEP levels.
- From an end-user standpoint, the results of the MED calculation process are:
  - A "log" to which new changes in BMEE MED states pertaining to the interval being calculated are added each time the process is run.
  - A table that contains all BMEE Days that are MEDs.
  - A table that indicates the values of the BMEE thresholds (for each year).
  - A flag at the level of each incident that indicates whether the incident is part of a MED.
  - There is also a general message log that keeps track of the steps being executed. The log is intended to be used by support personnel.
- The calculations involved in the MED processing are organized in a modular way, so that the timing of different execution steps can be adapted without major rework. By default:
  - The MED Thresholds (MED_THRESHOLDs) are determined on a yearly basis. One set of thresholds (one for each BMEE) applies for a whole year.
  - The MED_THRESHOLDs for a given year are normally determined based on the SAIDIs of the 5 prior years.
  - The process to determine/update Major Event Days (MEDs) is normally run once each day. At that time, SAIDIs for a given interval are determined and are compared to their corresponding MED thresholds. These calculations determine which days (BMEE days) over the interval are Major Event Days.

SAIDI and MED for a time interval are determined as follows:

- During the month of January, the MED thresholds used for the previous year are applied (that is, thresholds used from February to December of the previous year). SAIDI and MED are recalculated for the range of January 1 of the previous year until the current day minus a configurable number of days (specified in the MED_CALC_LAG_DAYS field in the MED_PARAMETERS table).
- During the months from February to December, MED thresholds for the current year are re-calculated (once), using the SAIDIs of the 5 prior years. SAIDI and MED are recalculated for the range of January 1 of the current year until the current day minus a configurable number of days (specified in the MED_CALC_LAG_DAYS field in the MED_PARAMETERS table).

8.2. Implementation Overview

The table below shows tables in the Archive database that exist specifically for the MED functionality.

Table 2. Archive Database Tables for MED Functionality

Table Name	Description	Populate/Update Approach
MED_PARAMETERS	Contains configuration parameters related to MED processing.	Initially populated by a script. Updated manually.
MED_DAILY_SAIDI	Contains daily SAIDI values.	Initial 5-year history values are populated when executing an offline script. Automatically updated when executing the corresponding MED calculations.
MED_THRESHOLD	Contains MED Thresholds (MED_THRESHOLDs)	Automatically updated when executing the corresponding MED calculations.
MED_LOG	Contains log entries that indicate BMEE day transitions between MED and Non-MED.	Automatically updated when executing the corresponding MED calculations.
MED_LOG_STATE	Contains state variables that the consumer of MED_LOG can use to determine which MED_LOG entries are new since the last time MED_LOG was read.	Automatically updated when executing the corresponding MED calculations.

Table Name	Description	Populate/Update Approach
MED_SERV_CENTER_BMEE	Contains mapping between SERVICE_CENTER and BMEE.	Initially populated by executing an offline script built from historical data provided by the project. This table can also be populated by querying the organizational hierarchy tables after those tables are populated. Typically, this table does not need to be updated.
MED_BMEE_CUSTOMER_COUNT	Contains BMEE customer count yearly average.	Initially populated by executing an offline script built from historical data provided by the project. Update of this table is project -specific.
MED_CALC_LOG	General purpose log pertaining to the MED processing.	Log message records are automatically added as the MED calculations are executed.
MED_DAY	Contains all BMEE Days that are Major Event Days.	Automatically updated when executing the corresponding MED calculations.
MED_EXCLD_INCIDENT_CAUSE	Contains incident cause codes for which the incident is excluded from SAIDI calculations.	Initially populated by executing an offline script built from historical data provided by the project. Updated manually.
MED_EXCLD_OUTAGE_TYPE	Contains incident outage type codes for which the incident is excluded from SAIDI calculations.	Initially populated by executing an offline script built from historical data provided by the project. Updated manually.
MED_EXCLD_DEVICE_TYPE	Contains Incident device type codes for which the incident is excluded from SAIDI calculations.	Initially populated by executing an offline script built from historical data provided by the project. Updated manually.

Table Name	Description	Populate/Update Approach
MED_BMEE	Contains the valid BMEEs. It is used for referential integrity checking of BMEEs, and for each BMEE (ID), contains its corresponding name.	Initially populated by executing an offline script built from historical data provided by the project. This table can also be populated by querying the organizational hierarchy tables after the tables are populated. Updated manually.

The table below shows stored procedures in the Archive database that exist specifically for the MED functionality.

Procedure Name	Description
MedCalcDailySaidi	Calculates daily SAIDI for the time range passed as an argument.
MedCalcThreshold	Calculates MED thresholds (MED_THRESHOLDs). $$MED\_THRESHOLD = e^{\left(\frac{\sum \ln(SAIDI)}{Number\ of\ SAIDIs} + 2.5\ STDEV\ (\ln(SAIDI))\right)}$$
MedCopyThresholdFromPrevYear	Copies MED_THRESHOLDs to the target year passed as an argument from the previous year's values.
MedCalcMed	Determines which days are BME days over the interval specified using the year provided for the threshold.
MedPurge	Purges the logs (MED_CALC_LOG, MED_LOG) based on the history retention values defined in MED_PARAMETERS.
MedPeriodicProc	This procedure normally runs periodically (for example, daily) but could be manually executed anytime. It does the following: • Sequences execution of calculation of daily SAIDI (updates MED_DAILY_SAIDI), MED thresholds (updates MED_THRESHOLD), and MED (updates MED_DAY and MED_LOG). • Executes the purge procedure.

8.2.1. MED Installation and Scheduling of Periodic Execution

Installation of the database objects (tables and stored procedures) related to MED is performed automatically when installing the Archive database schema.

After the Archive database schema is installed, do the following to enable MED calculations.

1. Populate the following tables with project-specific data:
    - MED_SERV_CENTER_BMEE (can be obtained from the organizational hierarchy tables)
    - MED_BMEE_CUSTOMER_COUNT
    - MED_EXCLD_INCIDENT_CAUSE

- MED_EXCLD_OUTAGE_TYPE
- MED_EXCLD_DEVICE_TYPE

2. Populate the MED_THRESHOLD table with the threshold values (MED_THRESHOLDs) for the past and current years. Alternatively, the MED_THRESHOLD values can be calculated from the 5-year history of SAIDI values, if these values are provided.
3. Configure periodic execution of the MED processing (the main stored procedure). For the configuration steps for SQL Server and Oracle, see Configuring Periodic Execution in SQL Server and Configuring Periodic Execution in Oracle.

8.2.2. Configuring Periodic Execution in SQL Server

1. Open the MS SQL Server Management Studio.
2. In the Object Explorer, connect to your database instance.
3. Expand the "SQL Server Agent" display, right-click on "Jobs", and then select "New job".
4. Select the "General" page of the "New job" definition.
5. Enter a name (for example, "MED Processing").
6. Select the "Steps" page of the "New job" definition.
7. Click "New" to create a new step.

   The New Job Step display opens.
8. Enter a name (for example, "Execute MED main procedure").
9. Select the "Transact-SQL script" type.
10. Select the database (typically "ARCHIVE").
11. In the Command window, enter "EXEC ARCHIVE.MedPeriodicProc".
12. Click OK.
13. Select the "Schedules" page of the "New job" definition.
14. Click "New" to create a new schedule.

    The New Job Schedule display appears.
15. Enter a name (for example, "Daily MED Processing").
16. Select the "Recurring" schedule type.
17. Set the frequency entries for the execution (a daily periodicity is desired). Set the time of day, the start date (normally, the current day), and select "No end date".
18. Click OK.
19. Click OK for the New Job.
20. Make sure the SQL Server Agent is started (the color changes to green).

8.2.3. Configuring Periodic Execution in Oracle

The periodic batch execution can be configured using Oracle's DBMS_SCHEDULER package. For example, a 7 PM daily execution can be configured from SQL*Plus (logged into the ARCHIVE user) by issuing the following command:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
   job_name           => 'MED_PERIODIC_PROCESSING',
   job_type           => 'STORED_PROCEDURE',
   job_action         => 'Medcalc.MedPeriodicProc',
   start_date         => '2018-OCT-08 19.00.00', /* Use date of next occurrence*/
   repeat_interval    => 'FREQ=DAILY;INTERVAL=1',
   comments           => 'An optional comment');
   COMMIT;
END;
/

BEGIN
  DBMS_SCHEDULER.ENABLE ('MED_PERIODIC_PROCESSING');
  COMMIT;
END;
/
```

8.2.4. Diagram: MED Calculation Implementation Overview

The below image shows the diagrammatic overview of the tables, columns, and stored procedures that support the MED functions.

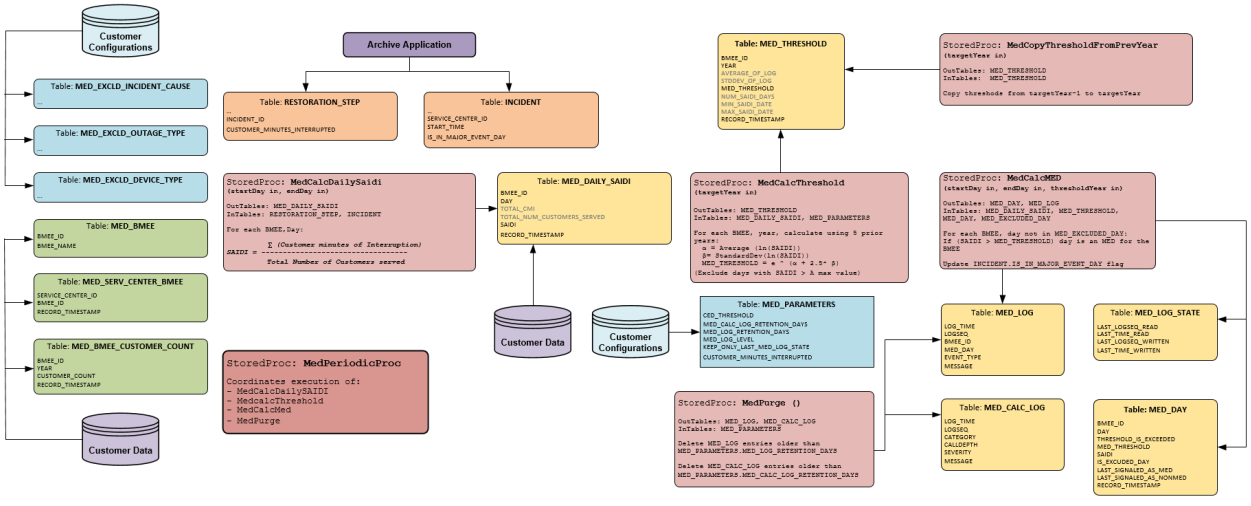


Figure 13. MED Calculation Implementation Overview